Slot-Chain: A Preconfirmation Protocol Built on Per-Slot Signature Chains

v2 Design Specification

August 2026

Abstract

This document specifies the successor design (v2) of the Taiko preconfirmation protocol. L2 time is divided into one-second slots; a lookahead schedule, published an epoch in advance, assigns each slot to a unique builder. The builder produces a block and signs it, and because each signature covers the parent hash, the not-yet-landed blocks form a *signature chain*. Submitting such a chain to L1 together with a validity proof is called *landing*: an aggregator elected by a perpetual auction performs it in the normal case, but anyone may perform it when the aggregator is absent, because a block's authority comes from its builder's signature rather than from whoever lands it. Finality is given by the proof system together with a deterministic *settlement window*: among the candidates that have landed and been proven, the heaviest one is finalized when the window closes at a predetermined L1 height. We give the trust model, the liveness accounting, the slashing rules and the parameter geometry, and we state the accepted residual risks and the remaining open items explicitly rather than eliding them — including the deliberate absence of a censorship-resistance floor, which this design does not provide.

Contents

Scope. *This document specifies the Taiko preconfirmation protocol: L2 time is divided into one-second slots, a lookahead schedule assigns each slot to a single builder, each builder signs the block it produces over the parent hash so that unlanded blocks form a signature chain, and that chain is carried to L1 with a validity proof. It supersedes an earlier design line built on a perpetual auction with epoch-level determination, which §10 compares against.*

How to read it. *§0–§11 with Appendices A–C are the specification proper and can be read in order. A reader after the trunk alone can get a full picture from the §0 one-page summary together with the §8 liveness accounting table. Residual risks and accepted costs are stated where they arise rather than collected at the end; open items are listed in §11.*

0 One-page summary

In one sentence: L2 has one slot per second, and the lookahead — which states "who holds the right to produce a block in this second" — is published two epochs in advance. The builder whose turn it is produces a block and signs it, and the signature covers the parent block hash; the blocks, which are not yet on chain, are thereby strung into a signature chain. The act of packing this chain onto L1 together with a validity proof is called landing: in normal operation the aggregator produced by the auction is responsible for it, but when the aggregator goes offline anyone may do it — because a block's authority comes from the builder's signature, not from the lander.

The complete path from block production to finality:

This structure solves three things at once:

1. **Second-scale liveness.** A builder going offline costs only its own slot (about 1 second), and the next builder simply continues producing blocks on top of the last visible chain head. There is no window in which "one participant goes offline and the chain stops for tens of minutes".
2. **No L2 consensus is required.** "Who should produce the block in this slot" is decided by the lookahead, and "whether this block is legal" is decided by pure-function rules inside the proving circuit (verify the signature, verify the lookahead assignment, verify the link) — anyone reading L1 computes the same answer, with no voting and no arbiter.
3. **Landing carries no privilege, and the aggregator is not a single point of failure.** The signature chain carries its own authority; landing is merely transport. If the designated aggregator times out without landing, anyone may pick the signature chain up, land it, and collect a reward out of the aggregator's bond.

What is kept and what is removed (relative to v15):

Kept	Removed
The perpetual auction skeleton for the aggregator seat	Epoch boundary commitments (EBC) and the once-per-epoch irreversible determination
The per-block signature chain and the two-tier parent rule (new here, not inherited from v15)	The default derivation rule (§6.8 default derivation)

Kept	Removed
—	**The forced-inclusion queue** — removed wholesale by the design decision (simplification), together with forced-only blocks, P_forced, the dual queues and C_force/C_bridge/F_delay/H_force
The "party at fault pays, permissionless substitution" model (fault-paid permissionless fallback)	Total Anarchy mode (§8) — landing itself can already be backstopped permissionlessly, so it is no longer needed
The tiered bond / slashing concept	K_empty, availability certificates (AC), the H_cancel cancellation cascade, and the rest of the machinery supporting the epoch determination
The per-block anchor approach to L1→L2 bridging	Handover and seat-switching arbitration mechanisms (replaced by circuit rules)

Timeline illustration (slot = 1 second, epoch = 384 slots) — going offline leaves only a gap, and the chain does not stop:

The relationship between landing and fallback (details in §6): anyone may take a contiguous segment of the signature chain plus a validity proof and land it on L1 in a single transaction, referencing block bodies already published to L1 as blob data (§6.1), making it a candidate; the heaviest proven candidate within the settlement window W_settle is final at the window close (§5.6). When the aggregator times out without landing, anyone may land, and the reward is paid out of the aggregator's bond (§6.3).

1 Design goals and non-goals

Goals:

- **[G1] Second-scale continuity of ordering.** If any single participant (a builder or the aggregator) goes offline, the interruption of ordering service perceptible to users lasts at most a few slots (seconds), not a few epochs (minutes to hours).
- **[G2] No L2 consensus.** The lookahead, legality and canonicality are all pure functions of L1 data and circuit rules; L2 nodes neither vote nor arbitrate among themselves. This inherits the spirit of v15's I1: derivation is a deterministic function of L1 data.
- **[G3] Landing carries no privilege.** Moving an already-signed chain onto L1 requires no identity whatsoever; the designated aggregator is merely an efficiency arrangement (avoiding gas competition and duplicated proving), not an arrangement of power.
- **[G4] Proof plus a deterministic window is finality.** A landed batch carries a validity proof, which L1 verifies on the spot, and the batch becomes a candidate; at the close of the settlement window the heaviest proven candidate is finalized — there is still no optimistic period, no fraud proof and no arbitration, and there is no intermediate "proposed but unproven" state, so the entire body of machinery built around such an intermediate state (sealing deadlines, cancellation cascades, and most of the timing exemptions) is unnecessary.
- **[G5] (withdrawn) An inclusion floor is no longer a goal of this design.** This design decided to remove forced inclusion wholesale, for simplicity. There is therefore **no** entry path to L1 that does not depend on builders: whether a transaction gets included depends on whether the scheduled builder is willing to include it. If the entire set colludes to censor an address, the protocol offers no remedy whatsoever, and that address's funds have no

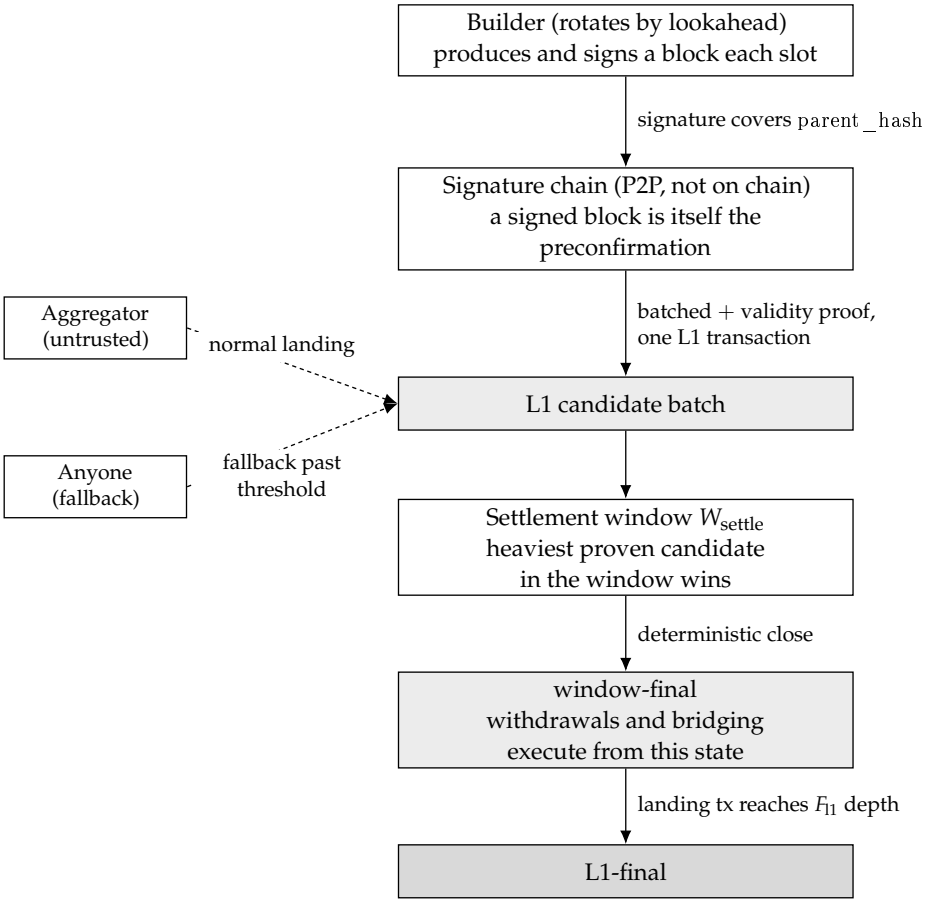

Figure 1: The complete path from block production to finality: builder signatures produce a signature chain that never touches the chain, anyone may land it on L1 together with a validity proof to become a candidate, and once the settlement window closes it becomes window-final and then L1-final. Landing carries no privilege, so the aggregator and the fallback lander are interchangeable.

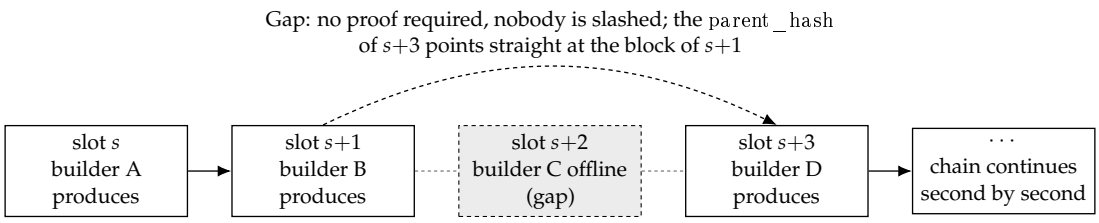

Figure 2: A single builder going offline leaves nothing but a one-slot gap in the timeline: the next builder takes the last visible block as its parent and carries on, so the chain never stalls.

exit path independent of the builders. This is an explicitly accepted cost; see the all-cartel row of §8 and the trust model below. One thing closes along with it: the nonce-preemption gap that a forced path would have had. With no forced path, the gap has nothing to apply to, and item 12 of §11 is retired with it.

- **[G6] A low barrier to entry.** The builder bond is priced at the order of magnitude of "the extractable value of a single slot", far below the full collateral for one seat in v15.

Trust model (normative):

- **Builders: honest majority.** We assume that within each lookahead window (on the order of an epoch), most of the scheduled builders honestly follow the protocol (producing blocks on time, broadcasting block headers and block bodies, and extending the highest available chain head).
- **Aggregators: untrusted.** An aggregator may misbehave arbitrarily — delaying, truncating, choosing among forks, colluding with a minority of builders — and the protocol must, under such conditions, still hold the bounds given in the §8 table. The scope of those bounds: the bounds in §8 are liveness bounds (advancing finality, seat turnover); the exposure of preconfirmation safety to collusion between the aggregator and malicious builders — isolation without slashing, a single-block deep skip of depth $\leq \mathrm{G\_max} - 1$, and a minority coalition relaying a sparse branch to a depth that can reach the length of the unlanded tail ($\approx \Delta\_\mathrm{lag}$ + fallback response) — is the safety residual risk explicitly priced in §5.4, and it falls outside the "bounded" promise.
- **L1: safe and live.** The standard assumption of every L2. Added to it is one assumption that this design explicitly acknowledges, the Bounded-Inclusion Assumption (a fixed $\mathrm{W\_settle}$ close and the $\mathrm{D\_anchor\_max}$ freshness deadline cannot be shown robust from "it will eventually be included" alone): a candidate or landing transaction that pays a sufficient fee is included within $\mathrm{T\_include,max}$ L1 blocks, and the parameters satisfy $\mathrm{T\_include,max} < \min(\mathrm{W\_settle} - \mathrm{P\_prove,max} - \mathrm{margin},\ \mathrm{D\_anchor\_max} - \mathrm{D\_anchor} - \Delta\_\mathrm{lag,final} - \mathrm{P\_prove,max})$ (the setter relations of §11). Targeted L1 censorship exceeding that bound is out-of-model — in that case a better candidate may miss the close and a recovery block's anchor may expire — and every statement that relies on "robustness under temporary censorship" (including the recovery bound of §6.4) is uniformly conditional on this assumption. This gathers the same timing dependency, previously implicit, into a single stated assumption; it is not newly introduced trust.

The honest majority is an economic and social assumption, not one enforced by the protocol : the draw is weighted by bond and $\mathrm{w\_max}$ is no defense against Sybils (§3.2), so a single capital holder could in principle buy up the majority of the weight and push the system out of the model — of the same kind as the honest-majority assumption of any PoS chain. This design does not claim that the builder role is entirely trustless or entirely permissionless; this is the trust model explicitly accepted (2026-08-25), and what is bought in exchange is not paying the cost of protocol-level DA to cover collusion by all participants (the cost identity is given in Appendix A-3). Each promise is calibrated accordingly: the in-model bounds are given in the §8 table; out-of-model, With no forced inclusion, nothing is guaranteed. A note on the acceptance criterion: if the acceptance criterion is "no role may be trusted, everything fully permissionless", this design neither satisfies it nor attempts to — the acceptance criterion is the trust model accepted in this section; the user-facing promises, graded into three tiers, are

stated in §5.4.

How much of the assumption each mechanism actually consumes (written out, to keep the assumption from being quietly strengthened):

- Finality liveness (fallback landing always having a chain available to land) requires only ≥ 1 honest builder per window (§6.3) — weaker than the assumption, so there is margin;
- The frequency with which preconfirmations are isolated (skips, body withholding) is bounded by the share of dishonest builders — this is where "majority" is used, and it is load-bearing there: skipping is profitable for the skipper, not a behavior that extinguishes itself for being unprofitable;
- Collusion of the entire builder set together with the aggregator (block bodies kept private, the chain landable by no one) lies outside the model: the protocol provides no finality guarantee in that case. After This design does not include forced inclusion, this case has **no defense in depth left**: the former guarantee that "even if the assumption is broken, forced transactions and bridge messages still operate independently on L1 data alone" no longer holds, and a user's inclusion and exit depend entirely on the honesty of the scheduled builders;
- **Preconfirmation safety for the tail additionally depends on the completeness of the lander's view** : a lander that is honest but eclipsed or blinded will land the wrong tail and revoke the preconfirmations of a heavier tail that it never saw. The mitigation is "landing depth $\geq$ propagation settling" (§5.2), which compresses this risk down to "equivocation (slashable) or eclipse (the P2P layer)"; together with "the honest majority is an economic assumption" and "liveness under a Byzantine aggregator is conditional on fallback being economically viable", it belongs to the explicitly stated non-pure-protocol dependencies.

Non-goals (not addressed in this version):

- Per-transaction fair exchange: the preconfirmation receipt is block-level (§5).
- User restitution: slashing is a deterrent, not insurance.
- Resistance to deep L1 reorgs: as with any L2, a deep L1 reorg can roll back L2 state.
- A built-in prover market: the aggregator either brings its own proving capability or procures it.
- **Collusion of the entire builder set plus the aggregator**: under the trust model above this is an out-of-model situation. With no forced inclusion, there is neither a finality bound nor any defense in depth for entry or exit.

2 Roles

- **Builder**: the set of addresses that have registered with the L1 contract and posted a bond. When the lookahead brings a builder's own slot around, it produces a block, signs it, and broadcasts it on the P2P network. The builder is the preconfer: the block it signs is itself the preconfirmation commitment for the transactions inside that block.
- **Aggregator**: the service provider that holds the sole seat by way of the perpetual auction. It has exactly one duty: to pack the signature chain into a batch, generate a validity proof, and land it on L1. It does not build, does not order, and holds no discretionary power over

content — the only residual discretion is "which prefix to land and when to land it", and both are constrained by deadlines and by permissionless fallback (§6).

- **Fallback lander**: anyone. A fallback lander appears once the aggregator has timed out and is paid under the fault-paid model. This is not a registered role.
- **User**: submits transactions to builders in exchange for preconfirmations. A censored user has no in-protocol remedy.
- **L1 contracts**: the builder registry, the aggregator auction, the candidate entry point (which verifies proofs), and the slashing entry point.
- **The proof system**: the circuit that verifies "this segment of the chain is legal and its state transition is correct". It is the arbiter of this design.

3 Time and scheduling

3.1 Slots and epochs

- **slot = 1 second**. The time unit of L2. There is at most one block per slot, and a block's timestamp is identically the starting second of its slot — the timestamp is not chosen by anyone, it is fixed by the lookahead (which eliminates the entire class of problems around the origin of timestamps found in Appendix C of v15).
- **epoch = 384 slots** (the current value is retained, aligned with L1's 32×12 seconds). In this design an epoch is merely a unit of measure (the granularity of lookahead windows and of fee settlement); it no longer carries the mechanism-level meaning of "one determination per epoch".

3.2 The lookahead

- The lookahead is a mapping $\mathrm{lookahead(slot)} \to \mathrm{builder\ address}$ that gives, for every future slot, the unique holder of the right to produce a block.
- **Computation (a pure function):** drawing entropy separately for each slot is not compatible with a lookahead horizon of useful length: the per-slot randomness at the far end of the horizon is not yet final at the moment the schedule must already be fixed. The seed is therefore drawn once per window: $\mathrm{lookahead(slot)} = \mathrm{draw(hash(seed(window),\ slot),\ registry\_snapshot(window))}$ — the seed is taken once per window, not once per slot:
  - **Window partitioning (if "window" were read as a sliding horizon, the same slot would receive different lookahead assignments at different moments, destroying the determinism of G2)**: a window is a fixed aligned partition: $\mathrm{W(slot)} = \mathrm{floor(slot\ /\ W\_size)}$, $\mathrm{W\_size} = 384$ (one L2 epoch, and — since an L2 slot is one second and an L1 slot twelve — exactly one L1 epoch of wall-clock time); the starting slot of window W is $\mathrm{W} \times \mathrm{W\_size}$, and "the L1 slot corresponding to that start" is converted by the genesis mapping $\mathrm{L1\_slot(s)} = \mathrm{GENESIS\_L1} + \mathrm{floor(s\ /\ 12)}$ (an L2 slot is 1 second, an L1 slot is 12 seconds). The lookahead assignment of any given slot has the same value for every observer at every moment.
  - **A unique snapshot height**: the snapshot height of window W is $\mathrm{H\_snap(W)} = \mathrm{the\ L1\ slot\ corresponding\ to\ the\ start\ of\ W} - \mathrm{D\_snap}$, where D_snap = the lookahead horizon $\mathrm{H\_look}$ (2 L1 epochs) + the finality distance $\mathrm{F\_final}$ (2 L1 epochs) + margin (1

L1 epoch) = 5 L1 epochs (the earlier $D\_snap = 3$ epoch omitted the horizon term: the lookahead is required to be computable H_look in advance, so the start of the furthest window is ≈ now + H_look, and its snapshot point = now + $H\_look - 3$ epoch = now $-$ 1 epoch, which is not yet finalized and can be reorged). The setter invariant: $D\_snap \geq H\_look + F\_final + margin$. It follows that at every moment at which the lookahead is required to be already determined, $randao\_source(W) = H\_snap(W) \leq$ finalized L1 head $<$ start of W — there is only one seed, it holds simultaneously for all slots in the window, and the lookahead is fixed before it is used and is unaffected by L1 reorgs.

  - **The seed**: the RANDAO of the L1 epoch containing $H\_snap(W)$ (provable in-circuit via the EIP-4788 beacon root); the variation between slots is derived by $hash(seed, slot)$, so no independent per-slot entropy source is needed.
  - **The registry snapshot**: the registry state at that same $H\_snap(W)$; the entry and exit delay (§4.1) guarantees that the snapshot covers the whole window. The circuit's lookahead check for every block in the window uses $H\_snap(W)$ uniformly, independently of each block's own anchor block.
  - **The draw**: deterministic bond-weighted sampling (independent for each slot). The weight of a single address is capped at w_max (initially 20%). An honest characterization: w_max only prevents concentration in a single address and is no defense against a Sybil that splits its bond — large capital need only split across several addresses to recapture the weight, and it costs not one cent more. The real upper bound on concentration is economic (buying weight requires genuinely locking up bond), the same trade-off as v15's acceptance that "the highest bidder can keep on winning". The role of w_max is operational hygiene; it is not a promise of decentralization.

- The full codification of the computation above (the constraint at each step is in the comments; for a runnable version with property tests see $lookahead\text{-}model.py$, and the sampling operator is the candidate for settling item 18(b) of §11):

```
W_SIZE = 384 # lookahead window = 384 L2 slots = one epoch (fixed aligned partition)
H_LOOK = 768 # lookahead horizon
D_SNAP_L1 = 5 * 32 # snapshot delay (L1 slots); invariant D_snap ≥ H_look/12 + F_final + margin
W_MAX = 0.20 # cap on a single address's effective weight (operational hygiene; no defense against a bond-splitting Sybil)

def window_of(slot):
# fixed aligned partition: the same slot maps to the same window at every moment and for every observer
return slot // W_SIZE

def snapshot_height(w):
# unique snapshot height = the L1 slot corresponding to the window start − D_snap.
# the three terms of D_snap guarantee: before the lookahead is used, this height is already
# L1-finalized — neither the seed nor the registry is affected by an L1 reorg
return l1_slot_of(w * W_SIZE) - D_SNAP_L1

def seed(w):
# seed = the RANDAO of the L1 epoch containing the snapshot height (provable in-circuit via the EIP-4788 beacon root).
# taken once per window, not once per slot: all slots in the window share one entropy source
return hash("seed", randao_at(snapshot_height(w)))

def effective_weights(registry):
# effective weight = min(bond, w_max × total bond). capped once, no iterative renormalization —
# the excess is voided rather than redistributed; a single linear pass, circuit-friendly
total = sum(registry.values)
return {a: min(b, W_MAX * total) for a, b in registry.items}
```

```
def lookahead(slot):
# lookahead(slot) = draw(hash(seed(window), slot), registry_snapshot(window)).
# the registry snapshot and the seed are taken at the same height; the variation between slots is derived from the hash
w = window_of(slot)
reg = registry_at(snapshot_height(w))
r = hash(seed(w), slot)
return weighted_pick(reg, r)

def weighted_pick(registry, r):
# deterministic weighted sampling (the candidate operator for settling ğ11-18(b)):
# 1. addresses in a fixed order by registration index (an order intrinsic to the snapshot, preventing grind-
ing on the registered name);
# 2. prefix sums of the capped weights (fixed-point integer wei, no floating point);
# 3. x = r mod total weight; whichever prefix interval x falls into determines the address selected.
# probability of being selected = share of effective weight; in-circuit = one prefix-sum pass + one modulo + one lookup
eff = effective_weights(registry)
addrs = sorted_by_registration_index(registry)
x = r % sum(eff[a] for a in addrs)
acc = 0
for a in addrs:
acc += eff[a]
if x < acc:
return a
```

- **Lookahead availability.** A window's schedule becomes computable as soon as its snapshot height is L1-final, that is, as soon as $\mathrm{H\_snap(W)} \leq \mathrm{L1\ head} - \mathrm{F\_final}$. Since $\mathrm{H\_snap(W)} = \mathrm{L1\_slot(W \times W\_size)} - \mathrm{D\_snap}$, every window whose start lies at most $12 \times (\mathrm{D\_snap} - \mathrm{F\_final})$ L2 slots ahead of now is already determined. The guaranteed lookahead is therefore

$$12 \times (\mathrm{D\_snap} - \mathrm{F\_final}) = 12 \times (160 - 64) = 1152 \text{ L2 slots} \approx 19.2 \text{ minutes}$$

independent of $\mathrm{W\_size}$ — the window size cancels out of the derivation, because the tightest case is a window boundary falling exactly at the horizon, which can happen for any $\mathrm{W\_size}$. $\mathrm{H\_look} = 768$ slots (12.8 minutes) is the published floor that consumers may rely on; the remaining 384 slots are deliberate slack. Note that the horizon is a consequence of $\mathrm{D\_snap}$ and $\mathrm{F\_final}$ alone, not of the window size: shortening or lengthening the window does not move it. The temporal geometry of snapshots, windows and the lookahead horizon:

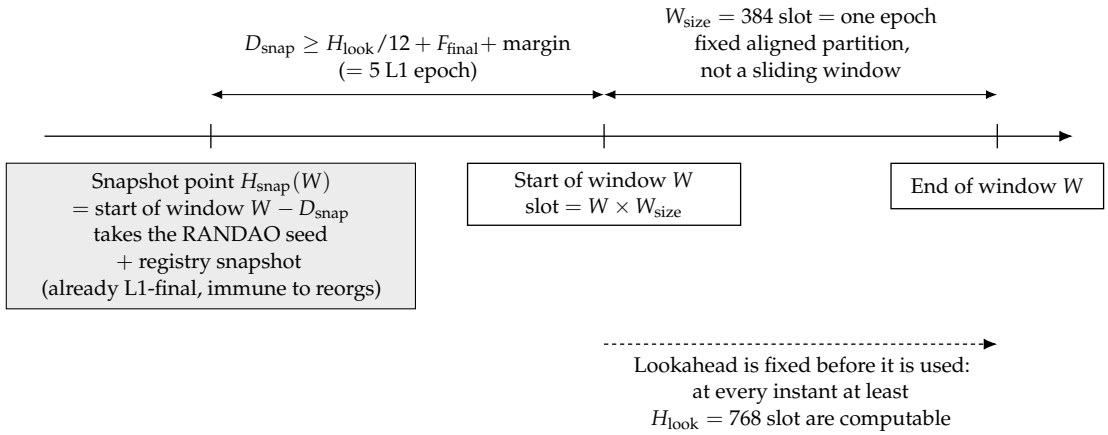

Figure 3: The time geometry of the snapshot point, the lookahead window and the lookahead horizon: entropy and the registry are frozen once and for all at D_{snap} before the window starts, so at every instant at least one full window of lookahead can be computed by anyone.

- **Why "the hash of the most recent proof" is not mixed in as entropy (the design decision; a deliberate divergence here — see Appendix A-1):** one and the same statement can generate an unbounded number of byte-distinct valid proofs (proof generation carries randomness of its own), so a proof hash is entropy that the prover producing it can re-grind at zero cost — mixing it in would amount to handing part of the control over the lookahead to the aggregator. Pure L1 beacon randomness can be biased by L1 proposers by roughly one bit each, but it is good enough for a use such as the lookahead and introduces no new controlling party. The principle: entropy comes only from the L1 consensus layer, and the moment at which its value is taken is earlier than the lookahead window.
- **A known cost, stated openly:** a public lookahead means that "who produces the block in the next second" is public, so a targeted DoS against a builder about to produce a block is theoretically feasible. Mitigations: a slot is only 1 second long (the attack window is minuscule), builders can use sentry or private networks, and a slot that is DoS'd merely turns into a gap (§5.3). Ethereum itself runs with the same public lookahead.

4 Builders

4.1 Registry

- Maintained by an L1 contract. Registration requires posting a bond $\mathrm{B\_builder}$, which serves two purposes (one account, two red lines):
  - **Equivocation slashing allowance** $\mathrm{L\_eq}$ (ETH). The pricing base is the cumulative exposure over a window, not a single slot : before a builder has been slashed for the first time and that slashing has taken effect (δ_slash in §4.3), it can pre-sign both forks in bulk for every slot it holds in the lookahead for the current window, so $\mathrm{L\_eq} \geq$ the maximum number of lookahead slots held by that builder within one exposure horizon $\times$ the worst-case extractable value per slot $\times$ a safety factor, where the exposure horizon = the maximum determined schedule + δ_slash + **one** $\mathrm{W\_settle\_max}$ + a margin for detection and submission latency. The $\mathrm{W\_settle\_max}$ term is required by the window scoping of §4.3: a slashing that becomes effective just after a window opens does not apply until the next one, so the signer keeps contributing blocks for up to a full settlement window after its bond is gone. The maximum determined schedule at $\mathrm{W\_size} = 384$ is 1535 slots, and $\mathrm{W\_settle\_max}$ = 150 L1 slots = 1800 L2 slots, so with $\mathrm{w\_max}$ = 20% and δ_slash = 64 the cumulative exposure is $0.20 \times (1535 + 64 + 1800) \approx 680$ slots — more than twice what the same formula gave before window scoping, and the price paid for removing the timing attack that keying on landing time allowed. Note that this must not be computed over a single window: a double-signer can carry on signing into the next window before the first slashing takes effect. The legitimate way to push this number down is to shorten the slashing-effectiveness delay and the detection latency, not to pretend that the exposure is only one slot. The concrete valuation method is calibration content under item 2 of §11.
  - **Nuisance buffer**: a small amount covering the handling cost of sporadic violations.
  - **The honest upper bound of** $\mathrm{L\_eq}$: $\mathrm{L\_eq}$ is priced against an estimate of the extractable value visible to the protocol; for value external to the protocol (bribes, cross-domain positions, downstream value on the bridge) the protocol can give no upper bound — so the equivocation deterrent is a "deterrent priced against an estimate", not a guarantee

against arbitrary external incentives. Any party that relies on commitments exceeding the size of the bond must price that residual risk itself (the same honesty clause as "deterrence, not compensation" in v15 §5.2; compensating users is a non-goal to begin with, §1). Evidence is idempotent: multiple pieces of equivocation evidence against the same builder trigger slashing only once (L_eq is confiscated in full, in one shot, and any later evidence is a no-op) — the total deterrence ceiling for a single builder is exactly L_eq, which is precisely why §4.1 prices it against the exposure horizon as a whole and why §4.3 uses δ_slash to freeze the builder's remaining slots into gaps as quickly as possible.

- **Entry/exit delay and bond retention period** : after an exit request, every slot already scheduled to the builder in the lookahead remains valid (it either produces a block or becomes a gap) — any lookahead entry that has already entered a window's snapshot $H\_snap(W)$ is valid throughout that window without exception, and does not lapse even if that address subsequently (after the snapshot height) exits the registry (the earlier blanket statement that "the lookahead never contains an address that is no longer registered" was ambiguous: an address that exits after the snapshot does still appear in the already-fixed lookahead, and the correct formulation is "its scheduled slots remain valid and are handled as either a block or a gap"); moreover, the actual unlocking of the bond must wait until all of that builder's slashing exposure has expired — setter invariant: **exit delay $\geq$ the distance to its last scheduled slot +** δ_slash **+** W_settle_max **+ a margin for detection and submission latency**, the same formula as the exposure horizon of L_eq above — the W_settle_max term covers the last settlement window into which a historical equivocation of that builder can still be introduced under the backfilling readmitted in §4.3. Otherwise a builder could equivocate in bulk shortly before exiting and reclaim its bond ahead of the evidence being submitted and taking effect, zeroing out the deterrent.
- Registry capacity cap N_max (initially 64): decentralized enough, while keeping lookahead verification inside the circuit cheap. Applicants beyond the cap compete for seats ranked by bond (detailed rules pending, §11).

4.2 Block production and signing

- The builder whose turn it is at slot s constructs a block header:

```
header = (chainId, slot, parent_hash, anchor, final_ref(tier (ii) blocks only, zero otherwise),
txs_root, state_root_claim(optional), coinbase, gas_used,...)
sig = the builder's signature over keccak(header)
```

Key fields:

- $slot$: the slot number of this block; $timestamp$ is uniquely determined by it and does not exist separately.
- $anchor$: the L1 block referenced by this block (the freshness baseline and the L1→L2 consumption point, §7; $m\_consumed$ in §7 is determined by the block header's anchor, so it must be a block-header field covered by the signature rather than a batch-level implicit quantity).
- $parent\_hash$: the hash of the parent block header. Having the signature cover parent_hash is a central element of this design: the choice of parent (including "whom to skip") is an explicit act that the builder has signed, not a fuzzy state that can be repudiated afterwards

. The link uses the hash of the parent block header rather than the parent's signature — the hash is structurally necessary (it defines the chain), whereas putting the parent's signature into the child block as well is redundant (the circuit verifies every block's signature anyway); it may serve as an evidence-packaging convention at the P2P layer, but it is not a consensus rule.

- **Signing-domain uniqueness invariant** : the canonical digest of the block header and the signature scheme must be one shared object — the direct verification by the L1 slashing contract (§4.3) and the validity verification in the circuit (§5.1) verify exactly the same bytes, the same hash, and the same curve. If BLS (via the EIP-2537 precompile) or a SNARK-friendly hash is adopted to reduce circuit cost, both sides must be switched at the same time. The choice of key system is therefore a blocking open item (item 6 of §11) that must be settled before the remaining parameters.
- The builder broadcasts (block header + signature + block body) over P2P. This signed block is itself the preconfirmation: the receipt given to the user = (block header, builder's signature, inclusion proof of the transaction) — the receipt contains the block header hash and $\mathrm{txs\_root}$, and is publicly forwardable evidence (used in §5.5).
- The decision procedure of the two-tier parent rule (the main statement and its exceptions are given in the next item):

- **The parent rule — two tiers, with the criterion being [the size of the gap] rather than the parent's landing status**: $\mathrm{parent.slot} < \mathrm{s}$, and the parent satisfies one of the following. The criterion is the gap $\mathrm{s} - \mathrm{parent.slot}$, evaluated over the structure of the signature chain (the gap + $\mathrm{parent\_hash}$) — the evaluation at signing time, at proving time and at landing time agree, and it does not depend on whether the parent "has landed / is already final" at this instant, a state that drifts with time:
  - **(i) Bounded-gap tier (normal block production)**: $\mathrm{s} - \mathrm{parent.slot} \leq \mathrm{G\_max}$ (the gap cap, initially 64 slot). The parent need only be a real earlier block in the signature chain ($\mathrm{parent\_hash}$ points at it), and its landing status is immaterial — not landed, landed but not final, or even already final are all admissible. Normal block production (including the next block after every landing) always takes this tier: after a landing, the next block attaches with a small gap to the (landed) chain head, and a gap $\leq \mathrm{G\_max}$ is legal, so "landed but not final" therefore constitutes no coverage hole at all (the criterion is the gap, not the landing status, so there is no class of parent for which "neither tier applies"). This tier guards against deep reorgs: $\mathrm{G\_max}$ caps the rollback depth of a single block, and a malicious builder that wants to orphan a deeper unlanded tail needs a relayed sparse branch (see the residual risk below).
  - **(ii) Unbounded-gap tier (recovery)**: $\mathrm{s} - \mathrm{parent.slot}$ is arbitrary — but the parent must simultaneously be [window-final] (its candidate has already closed in the §5.6 settlement window; the waiting bound = $\max(\mathrm{W\_settle}, \mathrm{F\_l1}) + \mathrm{D\_anchor}$) and [L1-final] (L1-final, i.e. its landing L1 transaction has reached $\mathrm{F\_l1}$ depth; the two-tier parent rule always takes L1-final as its criterion; for the unification of terminology throughout the document see open item 18(c) of §11). It is entered only when the gap > $\mathrm{G\_max}$ (tier (i) is unavailable, i.e. a true stall). Why a large gap may be unbounded: by §5.2 a final landed block can no longer be reorged, so building on top of it cannot reorg any landed block; it is therefore safe by construction and needs no $\mathrm{G\_max}$. This tier is the

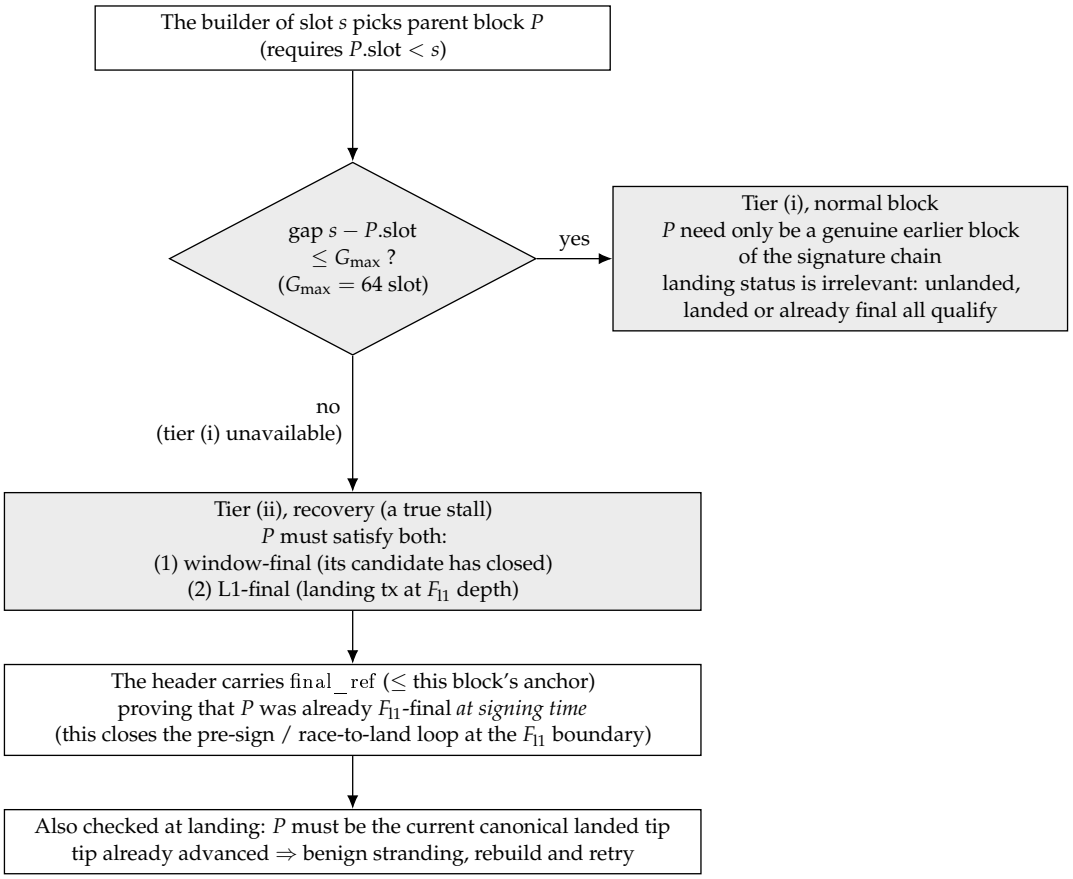

Figure 4: The decision procedure of the two-tier parent rule. The criterion is the size of the gap on the signature chain, not the landing status of the parent, so the rule evaluates identically at signing time, at proving time and at landing time; only a true stall, whose gap exceeds $G_{\max}$, drops into the recovery tier that demands a final parent.

only mechanism for recovery (it replaces the entire old re-anchoring episode machinery, see §6.4). The point of evaluation and the F_l1 wait: that the parent "is final and is the current canonical landing tip" is checked by the §5.6 landing rule at landing time, not at the moment of signing. Consequently, if a stall begins shortly after a landing and the current landing tip has not yet reached F_l1, the recovery block must wait for the current tip to become final (an extra $\leq F\_l1$, on the order of minutes) before it can land — building on an older final head would fork off the newer tip and be rejected by the §5.6 landing rule. A steady-state catastrophic stall (where the last landing became final long ago) incurs no such wait. Determinism likewise falls back on the landing rule: a batch may only extend the current canonical landing tip (§5.6), and a recovery block built on an old tip, if the tip has since advanced, is benignly stranded and can simply be rebuilt (there is no episode or deposit that could get out of order; during a true stall the tip does not move, so the recovery block is certain to land). F_l1 makes the predicate "final landed block" stable under shallow L1 reorgs (linked to L1 reorgs in §11).

  - A tier (ii) block header must carry an L1 reference attesting that [at signing time the parent was already F_l1-final] — closing the pre-signing loop at the F_l1 boundary (an attacker would pre-sign, then race to land it in the first L1 block at which the tip has just become final; this gives one iteration every F_l1, the lag never crosses its threshold, and the honest content has not yet aged to Δ_prop and therefore cannot challenge $\rightarrow$ a sparse chain of the attacker's own scheduled slots lands forever. The fix: the tier (ii) block header gains a new field $final\_ref$ = a reference to an L1 block, and the circuit / L1 entry point verifies that "this L1 block has already witnessed the parent reaching F_l1 depth", with $final\_ref \leq$ this block's anchor (available at signing time). The parent must therefore already be F_l1-final at signing time, and an unfinalized tip cannot be pre-signed; the race-to-land loop is broken (to land a tier (ii) block one must wait until the parent is truly final before signing, and by that time the honest content tail has had time to be proved and to land in the settlement window as a heavier candidate, which by §5.6 beats the sparse forced-only candidate outright). Normal recovery is unaffected: a recovery block is emitted only after its parent is final anyway.

- **Justification for the value $G\_max\_landed = \infty$ (the two-tier parent rule)**: tier (ii) has no cap because the reorg exposure through a landed head is naturally bounded by the length of the unlanded tail, and the tail length is bounded by Δ_lag — a tail older than Δ_lag has already been landed by the fallback, is final, and can no longer be reorged. "Unbounded" therefore does not increase the worst-case reorg depth (still $\leq \Delta\_lag$, the same bound as the existing residual risk in §5.4), yet it lets a stall of arbitrary duration be recovered in a single hop (see §8). A finite value loses at both ends: below the proving latency it can never catch up with a long stall, and set equal to Δ_lag the tightening is an illusion (nature already gives Δ_lag) —
- **"tail $\leq \Delta\_lag$" is conditional, not unconditional** : the claim above that "the tail length is bounded by Δ_lag" holds only when permissionless fallback can land an old tail within $\approx\Delta\_lag$, which is exactly the same conditionality as the one §6.3 attaches to the liveness bound under a Byzantine aggregator. If fallback is economically infeasible (nobody is willing to be a fallback lander) or the fallback landing transaction is itself censored on L1 (already listed in §6.3/§5.2), the unlanded tail will grow without bound as the stall continues, $> \Delta\_lag$; in that case $G\_max\_landed = \infty$ in tier (ii) lets a single malicious builder

orphan this over-long tail with one block (no longer requiring the coalition lookahead density of tier (i), which grows harsher as the tail lengthens), and a colluding lander that lands it thereby orphans honest preconfirmations reaching far beyond Δ_lag. The honest conclusion: the worst-case reorg depth of tier (ii) is $\leq$ the actual unlanded tail length, and it is $\approx \Delta\_\mathrm{lag}$ if and only if the fallback lands on schedule; when the fallback stalls, the depth grows with the stall, which is the same condition and the same root cause as the possibility in §6.3 that finality stalls indefinitely (no protocol-level DA + fallback economics / L1 censorship resistance). This is not a hole newly introduced by ∞ (a relayed coalition in tier (i) can equally orphan an entire over-long tail, merely at greater expense), but ∞ lowers the builder-side cost in that regime to "one person, one block", and this must be spelled out together with the optimistic annotation of $\leq \Delta\_\mathrm{lag}$. Under the same "someone is willing to land" condition, the §5.6 settlement window turns this kind of orphaning branch into a self-defeating move (a heavier honest tail candidate simply overrides it inside the window), but when "nobody provides the fallback / the fallback is censored on L1" the window may contain only malicious candidates — the conditionality is unchanged, only the failure mode shifts from "the malicious chain becomes final" to "the malicious chain becomes final in an uncontested window". This is still stated honestly.

- **Deep-reorg residual risk** : no protocol-level DA ⇒ L1 cannot see the unlanded tail ⇒ a colluding lander can always land a legal branch that excludes certain unlanded blocks, orphaning the preconfirmations they contain. The worst case depth = the unlanded tail length $\approx \Delta\_\mathrm{lag}$ + the fallback response time (once a tail is landed it is final). The builder-side cost under the two-tier rule: in tier (i) a single block reaches $\leq \mathrm{G\_max} - 1$, and going deeper requires a relayed sparse coalition; in tier (ii) a single block already suffices to offer a branch whose parent is a landed head and which orphans at most the entire unlanded tail. But both take effect only when landers collude — an honest lander follows the §5.2 fork choice, lands the heaviest tail and ignores such short branches (see the lander strategy in §5.2); the worst-case depth ($\leq \Delta\_\mathrm{lag}$) and the binding constraint that "lander collusion is required" are both unchanged, and all that changes is that tier (ii) lowers the builder-side cost of a fully orphaning branch from "a dense coalition" to "a single malicious builder". This is a residual risk explicitly accepted under the trust model of §1 (the cost identity in Appendix A-3), and it breaks only the tier-2 commitment (§5.4, which already declares that it can be revoked by a single malicious builder). $\mathrm{G\_max}$ (tier i) remains the governance knob that suppresses the feasibility of a sparse coalition.

4.3 Equivocation slashing (verified directly on L1)

- **Definition of the fault**: the same builder signs two byte-wise different block headers for the same slot.
- **Evidence and enforcement**: anyone may submit the two (block header, signature) pairs to the L1 slashing contract; the contract verifies the two signatures + same slot + different hashes directly on chain — no zero-knowledge proof and no arbitration period are needed, and once verification passes $\mathrm{L\_eq}$ is slashed, of which $\geq 80\%$ is burned and the remainder is paid to the submitter (following v15's burn-dominant principle, which prevents self-slashing arbitrage).
- **The point at which a slashing takes effect is a deterministic, slot-based rule** : if the timestamp of the L1 block that includes the slashing transaction corresponds to L2 slot $\mathrm{s\_0}$, then the slashing takes effect from slot $\mathrm{s\_0} + \delta\_\mathrm{slash}$ (initially δ_slash = 64 slot):

once it is in effect, every slot already scheduled to that builder in the lookahead is treated as a gap — the circuit rule: when a batch lands, the L1 contract hands the list of (slashed party, effective slot) entries it has itself recorded to the proof verification as a verified input; any block within the batch that is "signed by a party whose slashing is already in effect, with slot $\geq$ the effective slot" is invalid. Effectiveness is delimited by slot (rather than by each block's anchor block), which guarantees that all forks agree on the judgement of "who has already been slashed", so that no anchoring race exists. Without this clause, a slashed builder with a zero bond could still produce legal blocks (§4.1 and §5.1 would then contradict each other).

- **Effectiveness is scoped to settlement windows, so that no candidate can be invalidated after it is already in flight** : a slashing applies to a settlement window if and only if it became effective on L1 *strictly before that window opened*, that is, before the window's baseline was frozen (§5.6). The slash set is fixed at window open alongside the baseline, so every candidate in a window is judged against the same set, and the $\mathrm{header.slot} \geq \mathrm{effective\ slot}$ rule of the previous bullet is then applied within the window exactly as stated. A slashing that becomes effective mid-window applies from the next window; the delay is at most one $\mathrm{W\_settle}$.
- **Why not the L1 landing time of the candidate** : keying effectiveness to when a candidate lands is the obvious way to stop a zero-bond key from backfilling historical slots, and it was the earlier rule here, but it hands the attacker a far better weapon than the one it closes. The equivocator chooses when the evidence against itself is submitted, and $\delta\_\mathrm{slash} = 64$ slots is far shorter than the ten to fifteen minutes of proving plus inclusion — so it can reliably arrange for the effective time to fall after an honest candidate was built but before that candidate lands. It produces an ordinary block in its own scheduled slot, lets the honest chain extend through it, and then slashes itself: the heavier honest candidate is not outweighed but *invalidated*, every honest successor is stranded in cascade because its $\mathrm{parent\_hash}$ runs through that block, and a lighter candidate omitting the signer wins the window. The price of a tail-deep reorg would then be a fixed $\mathrm{L\_eq}$, and §5.6's guarantee that a heavier candidate simply overrides a malicious one would not apply, because weight is not what decides. It would also make a candidate's validity a function of the chain *and its L1 landing time*, contradicting the pure-function property claimed in §0. Window scoping removes the attacker's control: the rule is frozen before any candidate in the window lands, so the timing of the evidence changes nothing within it, and by the following window honest builders have re-rooted past the equivocator's block and there is no in-flight tail left to strand.
- **Backfilling of historical slots is therefore readmitted, and is accepted** : in a window where the slashing is in force, a zero-bond key can still sign a block whose $\mathrm{header.slot}$ precedes the effective slot and so passes the slot comparison. This grants it nothing it did not already have. Such a block must still occupy a slot the lookahead assigned to that very signer, bounded by $\mathrm{w\_max}$; it must still chain through $\mathrm{parent\_hash}$ into a real chain; and the candidate containing it must still win the §5.2 order against every honest candidate. What the slashing is for — removing the right to produce blocks from the effective slot onward — is fully enforced by the slot comparison. The one thing a zero-bond key genuinely gets for free is equivocation, since there is no further bond to take; but equivocation only multiplies candidates, each of which still needs its own valid proof and still has to be the heaviest, so it buys diversity rather than victory. The honest cost of

window scoping is stated plainly instead: a slashing bites up to one W_settle later than it otherwise would, during which the slashed key may still contribute blocks that win on weight. L_eq is priced against the value of a single slot, as §11 sets out, and no longer has to be priced against the value of an arbitrarily deep reorg.

5 The signature chain: validity, canonicality, preconfirmation semantics

5.1 Single-block validity (circuit rules)

A block is valid if and only if all of the following hold (all of them circuit-verifiable):

1. The signature is valid and the signer = $\mathrm{lookahead}(slot)$ (the lookahead is recomputed inside the circuit from L1-anchored data). There is no exception to this rule (after This design does not include forced inclusion, the P_forced forced-only-block exemption went with it — the right to produce a block belongs to the scheduled builder alone; the cost is that under a full cartel there is no longer an any-key escape path for block production, see §8 and §1). This replaces the old "three conditions" formulation that this design does not use;
2. $parent.slot < slot$, and the parent block satisfies one of the two tiers of the §4.2 rule(the criterion is the size of the gap, not the parent's landing status): (i) when the gap satisfies $slot - parent.slot \leq G\_max$, the parent may be any real earlier block in the signature chain (landing status is irrelevant); (ii) the gap has no upper bound, but the parent must be an L1-final landed block. $parent\_hash$ points to that valid parent. A tier (ii) block additionally requires a valid $final\_ref$: $final\_ref$ witnesses that the parent has reached F_l1-final and that $final\_ref \leq$ this block's anchor (§4.2); that the parent "is the current canonical tip" is still checked by §5.6 at landing time (every candidate is verified against the frozen window baseline; the two predicates are kept separate: at signing time finality is proved by $final\_ref$, at landing time canonicality is verified by the entry point);
3. $slot$ does not run past the range declared by the batch that lands it; the timestamp rule is automatically satisfied (= slot);
4. The execution of the transactions in the block is valid (ordinary state-transition verification); the anchor tx satisfies the freshness rule (§7) and satisfies the §7 causal-ordering invariant $anchor.L1\_timestamp \leq L2\ timestamp(slot)$; and inbound L1→L2 messages are consumed per §7 up to the [unified maximum prefix] — the longest prefix of the FIFO order that simultaneously satisfies "count $\leq C\_anchor$" and "cumulative declared gas $\leq$ the L1→L2 message gas share"; taking that longest prefix is valid, and only taking less than it is invalid(this rules out the indefinite censorship of "using a fresh anchor while processing zero inbound messages").

5.2 Canonicality and fork choice

- **The portion already landed on L1**: the canonical chain = the chain obtained by stringing together the window-final batches at the close of each settlement window (§5.6). Conflicting candidates within a window are superseded by the §5.2 total order, and the close settles the matter for life; only after the close is a candidate "permanently out".
- **The coordination convention for the unlanded portion (the P2P tail) — the ordering criterion is given uniformly by the "best-chain total order" of the next bullet**: honest

builders/nodes build on the chain head that is best under the total order of the next bullet (the old text's "most blocks + highest slot + hash" has been folded into levels 2/3/4 of the total order and is no longer listed separately, so that it cannot conflict with the total order). Positioning of its effect: this is an off-chain P2P coordination convention, not a circuit rule or an L1 entry-point rule — it constrains only the convergence of honest P2P nodes, and cannot constrain the fork choice of a malicious aggregator / fallback lander / private relay / L1 proposer; the real constraint on malicious landers is given by the §5.6 settlement-window competition (a worse candidate is mechanically overridden by a heavier one), and the bounding of the residual deep-skip exposure is still found in §5.4.

- **The best-chain total order — shared by three places: fork choice, landing strategy, and the §5.6 window candidate comparison**. Given a fork point F, the candidate chains extending F are compared, in the following order: **The object of comparison is a scalar triple, not a pairwise structural comparison.** (The old "look at the first differing block of the two chains to determine the lane" was a pairwise criterion, and a non-transitive cycle $\mathrm{A}>\mathrm{B}>\mathrm{C}>\mathrm{A}$ can be constructed: A=[X,a] vs B=[X,b1..b3] diverge at a/b1, so content wins; B vs C=[Y,c1,c2] diverge at X/Y, so block count decides; C vs A again diverge at Y/X, so block count decides; a cycle. Fix: the quantities compared must be each candidate's own invariant scalars, relative to the fixed baseline F and independent of the object of comparison). Each candidate chain extending F independently computes the triple $\mathrm{key} = (\mathrm{count}, \mathrm{tip\_slot}, \mathrm{tip\_hash})$:

  1. count: the number of valid blocks starting from F;
  2. $\mathrm{tip\_slot}$: the slot of the tip;
  3. $\mathrm{tip\_hash}$: the block-header hash of the tip(the smaller one wins; this is the last level and settles the tie-break. **The tie-break is reachable only through equivocation**, which bounds how often it can matter: two candidates with equal count and equal $\mathrm{tip\_slot}$ have their tips in the same slot, §5.1 requires $\mathrm{signer} = \mathrm{lookahead}(\mathrm{slot})$, and the lookahead is identical for every candidate in a window — so both tips are signed by the same builder, and a differing $\mathrm{tip\_hash}$ means two byte-wise different headers for one slot, which is precisely the §4.3 equivocation fault. Tip-only competition therefore arises only where a builder has equivocated. It does *not* follow that each such candidate costs $\mathrm{L\_eq}$: slashing is idempotent (§4.1), so a builder that signs many headers for one slot is slashed once and the marginal cost of every further tip is zero. That asymmetry is why §6.3 meters fixed-cost reimbursement on $(\mathrm{count}, \mathrm{tip\_slot})$ levels rather than per strict improvement. It also follows that an equal triple implies an identical tip header, hence an identical parent and inductively an identical chain, so the order is total on distinct candidates rather than merely on their keys. Comparison = the lexicographic order of the triple (count descending, tip_slot descending, tip_hash ascending) — a lexicographic order over scalars is naturally reflexive, total and transitive, so §5.6's "strictly heavier" relation and the winner at the close are thereby well-defined and independent of submission order. (The formalized property test is listed as item 18 of §11.) **On the absence of a lane component**: an earlier formulation put a lane (the class of the first block: a discretionary block = content lane, larger; a forced-only block = forced-only lane, smaller) in the leading position of the key, so that content chains would outrank forced-only chains and the whitewash of chain whitewashing would be resisted. With forced inclusion removed there are no forced-only blocks, all candidates are of the content class, lane is constant and draws no distinction at all, so the component is dropped.

The non-transitive cycle $\mathrm{A}{>}\mathrm{B}{>}\mathrm{C}{>}\mathrm{A}$ that a pairwise criterion would admit cannot arise: its root cause was that lane had been a pairwise criterion, whereas the three remaining components were always scalars belonging to the candidate itself. **A "frozen long tail" is no longer discarded**: a long tail whose head lags the wall clock by more than $\mathrm{G\_max}$ and can no longer be extended over P2P (no tier-(i) child, and, not being final, no tier-(ii) child either) is still the best chain among the candidates — the lander should land it first (honoring its preconfirmations) and only then resume producing blocks from its final tip (§6.4); landing it does not require extending it over P2P, only proving and posting it. The previously feared "deadlock between recovery and fork choice" thereby disappears: first land the best chain (frozen ones included) $\rightarrow$ then recover in one hop from its final tip.

- The final arbiter is still L1: disagreements are resolved once and for all by the §5.6 total order at the close of the settlement window. Equivocators are slashed (§4.3); blocks that forked innocently because of a network partition become orphans, and the transactions in them return to the mempool to be repacked.
- **The lander's tail-selection strategy**: when a lander faces several candidate tails, it applies the best-chain total order above to its own view and lands the best one. This no longer needs to be a "trusted obligation" — landing a worse chain gets it directly overridden by a heavier candidate inside the §5.6 window and its cost is spent for nothing, so landing the best chain is the only non-futile strategy; the protocol does not need to judge what a lander "ought to land", it only needs to let the heaviest proven candidate win. Δ_prop is retained as an off-chain strategy reference for the completeness of the lander's view(land the frontier where propagation has fully settled), and is no longer any kind of on-chain liability parameter. The handling of "there is a better tail but the lander did not see it": some lander did not see the heavier tail and landed a worse candidate? — that is fine: anyone who does see it lands the heavier tail together with its proof inside the same §5.6 window and directly overrides it. The incompleteness of a single lander's view is no longer a safety dependency(the earlier premise that "landing the wrong thing cannot be corrected" has been eliminated by the window); overall it is still required that at least one participant inside the window has a complete view and is willing to land — normal P2P propagation takes seconds and the pending frontier has long since spread everywhere, so this condition is naturally satisfied under the §6.3 fallback feasibility; network-wide eclipse-grade blinding is delegated to the §11 P2P spec.
- **The visibility assumption and the "economic only, not enforced" characterization**: this design assumes that, under normal P2P propagation, the aggregator (and any lander) can in most cases see the currently longest available signature chain — but the protocol cannot, and does not attempt to, force a lander to land the longest chain. The reason is structural: visibility of the chain can itself be taken away — the simplest example is a builder at the tail of the chain withholding the block it has just signed (§5.5 withhold-body / withhold-signature); the aggregator cannot obtain it and therefore cannot include it, and that is not a fault. Hence "land the longest visible chain" is a rational choice under economic incentives, not a slashable obligation: a lander that lands a shorter chain is not penalized, and the consequences run through only two economic channels — (i) being superseded by a heavier candidate inside the §5.6 window, having paid gas and proving fees for nothing; (ii) under the base-fee revenue-sharing rule below (§5.6/§6.5), the shorter the chain landed, the smaller the base on which the share is computed. the earlier attempt to turn "ought to

have landed but did not" into an adjudicable fault, and the refutation of that attempt, is precisely the cautionary example for this characterization.

- **A node's local chain-selection rule = the total order above**: the criterion an L2 node uses to dynamically maintain its "local best signature chain" is exactly the triple total order above; no second set of rules is needed. Equivocation is heavily slashed under §4.3, but one cannot assume that it does not happen — when it does, two signed blocks appear at the same slot and the chain splits in two; the two forks are still two individually valid candidate chains, and the triple yields a total order as usual (differing block counts are settled by count, equal block counts are arbitrated by $\mathrm{tip\_slot}/\mathrm{tip\_hash}$), so the convergence of node views is not interrupted by equivocation; slashing the equivocator (§4.3) is orthogonal to chain selection. Why the fee total does not enter the total order(a design trade-off): it was at one point considered to make "the sum of all base fees on the chain" one of the ordering inputs. The conclusion is that it cannot enter the canonicality criterion: fees are a quantity an attacker can manufacture by paying itself — a malicious builder that stuffs the block in its own slot with high-fee self-transfers can make a shorter chain beat an honest longer chain on the fee dimension, which amounts to buying out the honest majority's chain-selection outcome with capital (and if part of the fees flows back to the lander, the colluder's cost of manufacturing those fees is even partly refunded). The total order therefore keeps the number of genuinely signed blocks as its primary key (forging one requires stealing a builder's private key and cannot be manufactured); the fee total enters only the reward (the §6.5 base-fee sharing) and not the ordering — incentive alignment relies on money, canonicality determination relies on signatures.
- **Honest bounding**: before landing, an equivocating builder + one colluder willing to land its fork (the aggregator, or any lander inside the fallback window, or even a bribed L1 proposer) can make the "other" tail canonical and isolate the preconfirmations contained in it. This attack (i) requires one equivocation that L1 can slash directly (the cost = $\mathrm{L\_eq}$, priced by the exposure accumulated over the window, §4.1); (ii) has a depth bound = the length of the unlanded tail. The tail length is not "hard" capped by Δ_lag: $\mathrm{lag} > \Delta\_\mathrm{lag}$ only opens the authorization for fallback landing, it does not force any batch to be accepted; while an aggregator that withholds landing drags things out, the tail keeps growing throughout the fallback response time. The honest depth bound = Δ_lag + the fallback response time (one round of proving
  - landing $\approx$ 10–15 min $\approx$ 2–3 epochs), and it can be stretched further by sustained L1-level censorship of the fallback transaction — the latter being the most expensive censorship class, already priced in §6.3. This is a soft upper bound plus an explicitly stated response-time assumption, not a hard cap. Therefore the safety of a preconfirmation before landing is deterrence plus bounded depth, not absolute — the user-facing semantics of §5.4 are worded accordingly. Keeping the tail short (§6.3) is the principal means of shrinking the payoff of this class of attack.
- **Canonicality is still determined by L1 alone (G2) — but L1 does not adjudicate "which chain ought to be landed"; the settlement window selects it mechanically**: off-chain fork choice gives honest nodes convergence (the total order above), while "the best chain wins" is delivered mechanically by the §5.6 settlement window — L1 does not adjudicate "who ought to land what", it only lets the heaviest proven candidate inside the window become final. This design is therefore not "L1 enforces the longest chain" (that would require protocol-level DA in order to see the unlanded tail) but "L1 picks the heaviest

among the already-landed, already-proven candidates" — no DA, no ex post adjudication; it is the mechanical middle point chosen under the §1 constraints.

5.3 Gaps

- No valid block appears at slot s (the builder went offline, was DoSed, or its block body did not propagate), the chain is left empty at s, and the builder at $\mathrm{s}{+}1$ builds on an earlier chain head. Gaps are handled as a normal case, not as an anomaly: no proof of "why it is missing" is required, and nobody is slashed for a gap (producing a block is an opportunity, not an obligation — inheriting the principle of v15 §9.3, but now applying to every slot).
- The cost of a gap is absorbed by incentives: a skipped slot earns no fees, and users who hold preconfirmations settle accounts with that builder's reputation. Nuisance-level chronic absence is handled by the registry's liveness rules (an optional item, §11).

5.4 Deletion vs. gaps: what the signature chain fixes and what remains (stated explicitly)

The parent-block-hash chaining fix raised by the design has precisely the following effect:

- **Fixed: the aggregator's selective deletion.** Because a child block's signature covers $\mathrm{parent\_hash}$, pulling a block out of the middle of the chain breaks the chain for every successor and the proof necessarily fails. Over a signature chain, the aggregator retains only the power to take a prefix: land $[..\mathrm{m}]$ and leave $[\mathrm{m}{+}1..\mathrm{k}]$ outside the chain.(The deep-reorg path in which "a malicious builder starts a short alternative branch from an old point without equivocating, for the aggregator to fork-choose"; a single block is capped by $\mathrm{G\_max}$, but a relayed sparse branch can reach the entire unlanded tail)And the suffix that is left behind still attaches to the new L1 chain head, so the next batch (anyone's) can land it just the same — truncation is delay, not destruction. Together with the landing deadline (§6.3), a malicious truncation by the aggregator buys little.
- **What remains: the skip by an adjacent builder — and it is profitable.** Builder $\mathrm{s}{+}1$ builds on $\mathrm{s}{-}1$ and claims never to have seen the block at s — which cannot be falsified inside the protocol ("I did not receive it" is unprovable). previously extrapolated "the actor gains nothing" onto skipping, which was wrong: the skipper can repack the high-priority-fee transactions and the MEV from the skipped block s into its own block and still collect the entire revenue of its own slot — a skip is a profitable, non-slashable revocation of preconfirmations that a single untrusted builder can carry out on its own. Four points of honest bounding: (1) the choice is fixed by a signature and publicly visible ($\mathrm{parent\_hash}$ in black and white, plus the signed block at s circulating over P2P), so attribution has hard evidence; (2) the one-shot payoff of an adjacent skip is bounded by the fee/MEV transfer of one slot; **Deep skip / sparse private branch**: with a single block, one malicious builder can, in a single deep skip, affect at most $\mathrm{G\_max} - 1$ unlanded blocks; but a minority coalition that relays several blocks along a private branchcan affect the entire unlanded honest tail. The coordination convention as revised in §5.2 makes the honest network not follow such a fork, so it must be landed on L1 ahead of the others in order to take effect. The crux of the race: if the colluding lander is only a fallback lander, this really is a race against the honest batch already in flight (the window $\approx$ the normal landing cadence, and the fallback right does not open at all before $\mathrm{lag} > \Delta\_\mathrm{lag}$); if the colluder is the incumbent aggregator, then during normal operation it is the only routine lander and there is no race

to speak of. This combination is a security residual risk that this design prices explicitly, and its depth bound splits into three tiers according to the form of the attack:

- **Tier (i), single-block deep skip (unlanded parent)**: depth $\leq$ G_max $-$ 1 (a tail of roughly 1 minute), one malicious slot;
- **Tier (i), relayed sparse branch**: depth $\leq$ the unlanded tail $\approx \Delta$_lag + the fallback response, at the cost that the coalition must hold one scheduled slot in every G_max window of the tail (the density requirement becomes harsher as G_max is tuned smaller);
- **Tier (ii), single block via a landed head**: one malicious builder produces a block whose parent is a final landed head, and in a single step can supply a branch that isolates up to the entire unlanded tail, with no coalition density required. **Precondition**: for that branch to be landable on L1 by a colluding lander, its final parent block must at the same time be the current canonical landed tip (§5.6 only lets candidates that extend the current tip land) — that is, the current tip must itself already be final. In steady state the tip is usually already final, so the precondition usually holds and the threat is not weakened; the point here is only to state the residual risk accurately (when the tip is not final, this single-block attack must first wait for the tip to become final, which has the same origin as the F_l1 wait for recovery in §4.2). **The depth upper bound $\leq$ the length of the unlanded tail**, which lowers the builder-side cost of a fully isolating branch from "a dense coalition" to "one person, one block"; the worst-case depth and the binding constraint that "a lander must collude" both stay unchanged — this is the price accepted for G_max_landed $= \infty$, in exchange for one-hop recovery from a stall of arbitrary length (§6.4/§8). "$\approx \Delta$_lag" is a conditional upper bound: the tail length is $\approx \Delta$_lag only if permissionless fallback lands on schedule (the same condition as the §6.3 liveness bound under a Byzantine aggregator); when fallback stalls, or the landing transaction is censored by L1, the tail grows without bound along with the stall, and a single tier-(ii) block can isolate a tail far longer than Δ_lag — see that conditional argument in §4.2 for details. This is not unique to ∞ (a tier-(i) relay coalition can likewise isolate an over-long tail, merely at greater cost), but ∞ lowers the cost in that regime to one person and one block, and this must be stated honestly alongside it. The frequency of both tiers is $\leq$ the number of scheduled slots the coalition obtains, everything is attributable throughout (the parent choice is signed and on record, and the aggregator's truncated landing is public on L1), the only mechanical deterrents are seat value and reputation, and the protocol does not slash builders; but, for these isolating branches actually to be finalized they must in addition beat a heavier honest candidate inside the §5.6 settlement window — as soon as somebody lands the honest tail together with its proof into the window, the isolating branch is self-defeating; the residual risk narrows to "nobody lands the honest tail inside the window" (the fallback feasibility condition, §6.3). Complete structural elimination still requires protocol-level DA (Appendix A-3), which is not adopted, per §1. G_max is thereby a governance knob: tuning it up strengthens gap tolerance (§6.4), tuning it down raises the coalition's scheduling density required for a relayed sparse branch. The direct repacking payoff is still bounded by the capacity of the attacking block itself; the frequency of both kinds of skip is bounded by the fraction of dishonest builders — the "honest majority" of the §1 trust model is load-bearing exactly here (this is not conservative redundancy); (3) the harm to users is layered: pure "inclusion"-class commitments are usually honored in the next slot via repacking (the skipper has exactly the incentive to collect the fees of those transactions), while what is

genuinely broken are the "ordering / exact state"-class commitments; (4) the handling = public attribution + the registry's liveness/reputation rules (item 1 of §11) + users penalizing repeat offenders with their flow. This is a deterrence-grade residual risk, which this design accepts and states explicitly; structural elimination requires protocol-level DA / availability proofs (for the cost identity see Appendix A-3).

- **Corollary — preconfirmation semantics:** there are four ways in which the preconfirmation of a signed block can be broken:
  1. The builder that signed it equivocates and a colluding party lands the other fork — the cost is being slashed directly by L1 for $\mathrm{L\_eq}$, priced by the exposure accumulated over the window (§4.1); the isolable depth is $\leq \Delta\_\mathrm{lag}$ + the fallback response time(see the honest bounding in §5.2);
  2. Some later builder skips it — profitable for the skipper (fee/MEV transfer) and not slashable; publicly visible and fixed by a signature; an adjacent skip affects a single slot, a single-block deep skip affects at most $\mathrm{G\_max} - 1$ unlanded blocks, and a minority coalition's relayed sparse branch can reach the entire unlanded tail; it generally has to win the landing race— but no race is needed when the incumbent aggregator colludes; the frequency is bounded by the honest-majority assumption; pure inclusion-class commitments are usually honored via repacking (§5.4 above);
  3. The builder that signed it withholds the block body (§5.5) — the block can never be landed, the preconfirmation lapses as a matter of course, and the handling is likewise deterrence-grade;
  4. **A deep L1 reorg** — a residual risk shared by every L2. The correct statement is this: no single participant can annul a batch of preconfirmations "at no cost and silently" — every path either pays a slashing that L1 can verify directly, or leaves a public record of behavior with a signature on file; but "collusion that can afford the cost" can do it within a bounded depth. This is still stronger than the single seat of v15 (where the silence of one seat annulled the entire epoch's service with nobody able to substitute), but it is not an absolute guarantee and should not be advertised to users as an absolute guarantee. **The product must distinguish three tiers of commitment**: (1) inclusion intent — the transaction will get on chain, and this usually survives skips/withheld bodies (via repacking); (2) ordering/state preconfirmation — in this design it can be profitably revoked by a single malicious builder in the next slot, and is backed only by frequency bounds and public attribution, with no slashing; (3) landed finality on L1 — absolute (up to the L1 reorg depth). User-facing wording must be honestly labeled by tier, and tier 2 must not be advertised as a strong safety commitment.

5.5 Signing the header without publishing the body (withhold-body) — a known non-slashable misbehavior

- **The behavior**: the builder broadcasts (block header + signature) but withholds the block body. The consequences: later builders cannot compute its state and can only skip it; the aggregator cannot obtain the data and can never land it; the preconfirmations of users holding receipts lapse.
- **Structural self-healing**: a builder can only build on a parent block whose body it holds and whose state it can execute out — this is a physical fact of execution, and need not be

"required" as a rule. A suffix whose bodies are withheld will therefore be automatically bypassed as soon as the first non-colluding builder's turn comes (it has no choice but to attach to the last available block), producing an available fork that anyone can land. For a withheld-body chain to hold up the finality of the whole chain, every scheduled builder after it would have to keep colluding to share the bodies privately — that is the full-cartel case. Since This design does not include forced inclusion there is no backstop clause left in the protocol for it; see §8.

- **The characterization, stated explicitly**: this is not a slashable fault — "it has the body but does not publish it" is unprovable inside the protocol (the same undecidability as "skipping" has at its root). This design does not pretend to solve it; the handling is:
  1. **Preconfirmations before landing explicitly carry a DA assumption**: user documentation must state that "a preconfirmation before landing relies on best-effort availability of the block body on the P2P network, with no protocol-level DA guarantee";
  2. The receipt format (§4.2) leaves the victim user holding (block header + signature + inclusion proof) — public evidence of the withholding behavior, for use by the reputation system and by off-chain accountability;
  3. The withholder itself gains nothing (its block earns no fees and is bound to be skipped), so this is purely spiteful behavior, and deterrence rests on reputation and the registry's liveness rules (item 1 of §11).
- The honest conclusion of this design is therefore: **the slashable faults are the two classes that L1 can adjudicate mechanically — equivocation (§4.3) and landing-timeout strikes (§6.3)**; a lander landing a worse chain is not a "slashable fault" but "self-defeating"(inside the §5.6 window it is overridden by a heavier proven candidate and its cost is spent for nothing; there is neither a need nor an ability to adjudicate "what ought to be landed" mechanically); skipping and withholding bodies at the builder level remain non-slashable. The wording of §9 and §10 is updated accordingly. An honest inventory of the weak-enforcement class: withholding a body usually yields no direct protocol revenue; skipping (deep skipping included) can capture fees/MEV and can revoke ordering/state-class preconfirmations, and is not slashable — the only handling is frequency bounds (honest majority) and public attribution.

5.6 Settlement windows and heaviest-proven-batch finality (settlement-window finality) — mechanical, DA-free tail protection

The window lifecycle in a single diagram (in one-to-one correspondence with the pseudocode below and with the executable model of Appendix C):

- **Candidates and the settlement window — including the [baseline freeze + candidate versioning + close commit] state machine (a restoration of "the first to land wins permanently")**: let the current window-final head be F. Opening the window immediately freezes the baseline tuple $\mathrm{base} = (\mathrm{F},\ \mathrm{F.stateRoot},\ \mathrm{m\_consumed})$; every candidate inside the window is verified against the same frozen base (the proof's starting cursor/state root = base, consuming the same queue head), and passing verification records only that candidate's own end tuple $\mathrm{end\_i} = (\mathrm{tip\_i},\ \mathrm{stateRoot\_i},\ \mathrm{m\_consumed\_i})$ together with its §5.2 weight $\mathrm{key\_i}$ — provisional acceptance changes no canonical state (it does not advance cursors, does not switch parents, and does not execute bridge effects). At window close,

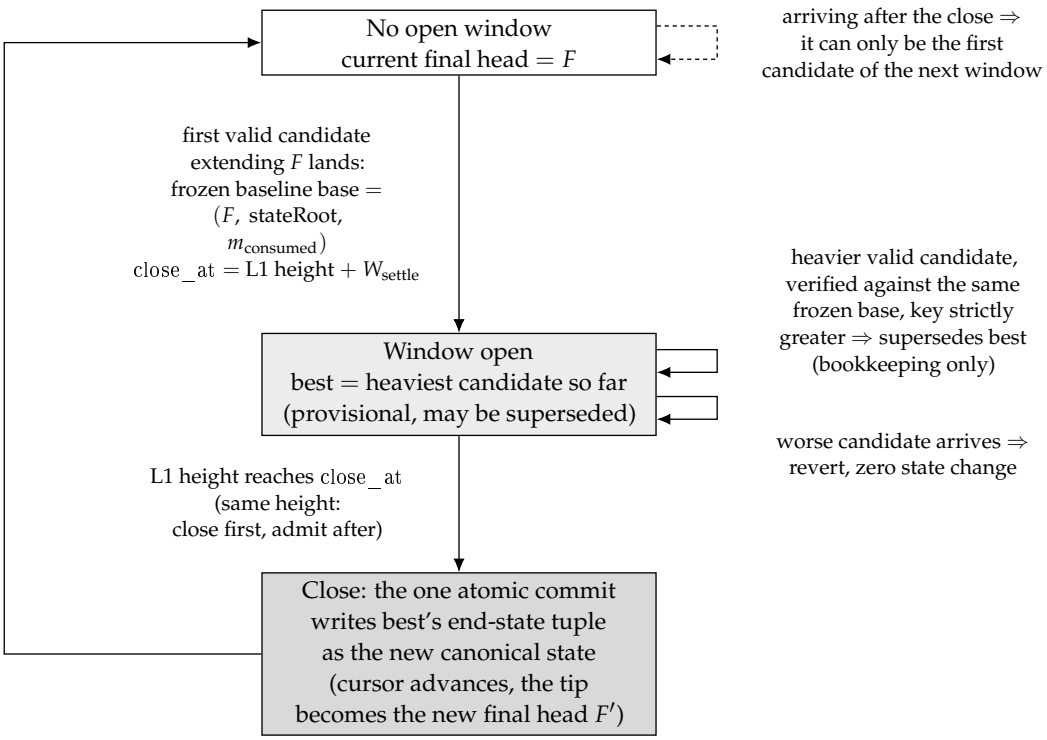

Figure 5: The life cycle of a settlement window as a state machine: opening the window freezes the baseline, every candidate inside the window is verified against that same baseline and only updates bookkeeping, and not until the close is the heaviest candidate's end-state tuple committed atomically as the new canonical state.

only the end tuple of the current heaviest candidate (the one with the largest key) is committed atomically, in one step, as the new canonical state; its tip becomes F', and the next window starts from the new base derived from F'. Pseudocode:

```
openWindow(F): base ← (F, F.stateRoot, m_consumed); best ← ⊥; close_at ← L1.now + W_settle
acceptCandidate(c): requires best=⊥ ∨ L1.now < close_at; # close precedes acceptance:
# at the same L1 height, closeWindow is evaluated first, acceptance afterwards;
# a candidate arriving after that point can only open the next window
if best=⊥: openWindow(current window-final head); # the first candidate opens the window
verify(c.proof, from = base); # all candidates share one frozen baseline
requires best = ⊥ ∨ key(c) > key(best); # ğ5.2 triple lexicographic order, strictly heavier
best ← (c.end, key(c)) # bookkeeping only, canonical untouched
closeWindow: requires L1.now ≥ close_at ∧ best ≠ ⊥;
canonical ← best.end; # the single atomic commit
credit(best.beneficiary,
φ_land × Σ base_fee(best)) # the base-fee share is booked at close (ğ6.5)
```

(What the baseline freeze covers is [the cursors and the state]; it does not include queue contents: the L1→L2 message queue on L1 is append-only, entries are referenced by global sequence number, and an entry's content is immutable once enqueued — so there is no need to "freeze a queue snapshot": two candidates starting from the same frozen cursor necessarily read the same content at every sequence position; new entries enqueued midway through the window are visible and determinate to any later candidate "whose blocks can reach them" (a heavier candidate legitimately consumes more), and the window state remains a pure function of L1 history — enqueue events included. In implementation this requires the queue commitment to be organized per-entry and append-only (a hash chain or an MMR), and proofs to reference entries by sequence number; a "single mutable queue root as of the moment of landing" must not be used as a public input — otherwise the public inputs of successive candidates within a window would drift. Model property P12. closeWindow may be called by anyone, or folded into the first transaction of the next window as a lazy close; the adjudication order at one and the same L1 height is normalized to "close precedes acceptance" — candidates arriving at or after the close_at height do not take part in this window and can only be the first candidate of the next window, and any implementation (including lazy close) must be consistent with that semantics (the replay of the executable model is exactly this semantics, Appendix C P5b/P6); the window state and best are both pure functions of L1 history, and under a shallow L1 reorg they roll back together with L1 — folded into item 9 of §11.) Stated honestly: this is a small window state machine (three actions: baseline freeze / candidate versioning / close commit) — far smaller than challenge/deposit/DA or the old episode, but It is a small state machine, and this document says so rather than claiming otherwise. The atomic advance rule for the m_consumed cursor of §7 is correspondingly re-read: both the starting check and the advance are relative to the window baseline and the close commit, and no longer to "whoever lands first advances".

- **Why "the heaviest proven batch" is by itself a mechanical protection of the tail**: an honest lander brings the complete honest tail (necessarily the heaviest under the §5.2 total order — under an honest majority, any branch that excludes honest blocks contains fewer blocks, see §5.4) onto L1 together with its proof, and it wins outright. What if a malicious lander gets ahead by landing a worse chain? Inside the window it is overwritten by the honest candidate; nothing is left on chain and no compensation has to be adjudicated, and the malicious party has paid L1 gas plus proving fees for nothing (a natural griefing cost that requires no design). Every "how do we prove that it should have landed at the time" difficulty of the earlier accountability-based attempts ceases to exist: the chain that ought

to land lands by itself and wins, and nobody needs to prove "that it once existed".

- **"Sniping" at window close is benign** : what if someone lands a slightly heavier candidate at the very last moment before close? Under the total order, "heavier = more genuine signed blocks" — forgery is impossible (forging would require stealing a builder's private key), so a heavier candidate necessarily contains more genuine preconfirmed blocks, and its winning is better for users (a more complete tail is finalized). The only party harmed is the earlier lander, in gas — an economic residual risk, not a safety residual risk. A fixed window therefore suffices, with no need for a chess clock or extensions (an extension would instead introduce gameable timing). Withholding a block body and releasing it late (a builder withholds the body $\rightarrow$ the fallback lands the visible tail $\rightarrow$ before close the attacker releases the body, lands the full tail and overwrites it) is the same story: the result is that a more complete tail is finalized, and the fallback lander that was "overwritten" loses gas — handled through the fallback-compensation path of §6.3 and listed as an economic residual risk.
- **The setter invariant for** $\mathrm{W\_settle}$: $\mathrm{W\_settle} \geq \mathrm{P\_prove,max} + \mathrm{T\_include,max} + \mathrm{margin}$ — this guarantees that, from window opening onward, anyone who sees a heavier tail has time to prove it and land it. An honest lander need not wait for the window: it starts proving as soon as it sees the tail, so under normal conditions the heaviest candidate is already on its way when the window opens. The steady-state cost is zero: with no competition the window contains only the aggregator's own single candidate, and nobody spends an extra cent — the competition mechanism is exercised only when the system is under attack.
- **The window state is a pure function of L1 history**: candidates are all L1 transactions, window opening and close are L1 heights, and the current best is the deterministic result of transaction-by-transaction comparison — any node that replays L1 obtains the same window state; under a shallow L1 reorg it rolls back together with L1 (folded into the atomic-rollback list of item 9 of §11, with no extra state machine).
- **Liveness and anti-spam** : superseding requires a strictly greater weight plus a valid proof, so a candidate spammer must pay a real proving fee every time and is capped by the length of the real tail — there is no livelock; a candidate arriving after window close is benignly stranded (it targets the old F, and it suffices to rebuild it onto the new $\mathrm{F'}$, the same stranding semantics as in §6.4). The candidate's beneficiary (the reward/refund address) is built into the proof's public inputs, so mempool copying cannot change where it points.
- **Explicitly stated residual risks (stated honestly)**: (a) temporary ordering swings during the window: the "current best candidate" inside a window may be superseded by a heavier one, so the temporary head that L2 nodes follow is mutable within the window — this corresponds precisely to the existing tier-2 preconfirmation semantics (revocable, decided by window close); it adds no new exposure and merely makes that exposure explicit; withdrawals and bridging execute only from window-final state (§6.1/§6.2). (b) **Fallback for excluded blocks**: if the heaviest candidate still excludes certain genuine blocks (for example, an attacker wins some window with a sparse branch built on enough slots of its own in the lookahead — under an honest majority this requires a coalition with dense lookahead scheduling, bounded as in §5.4), the excluded transactions return to the mempool as before and are repacked in the next window — the worst-case depth is still $\leq$ the unlanded tail, the same bound as the existing residual risk of §5.4, and C does not widen it. (c) **Finality +** $\mathrm{W\_settle}$: this is the real cost of the settlement-window design, and this is accepted explicitly (accounted for in §6.2).

6 Landing

6.1 Batches

- A batch = a contiguous segment of the signature chain $[\mathrm{m}{+}1..\mathrm{k}]$ (extending from the current window-final head m, §5.6) + references to the block-body data published as blobs (next bullet) + a validity proof that verifies all the rules of §5.1 together with the per-block state transitions.
- **Atomic landing + window finality** : a candidate is still created by one transaction — data references, proof and state root are submitted together and verified on the spot by L1, and publishing data creates no claim on anything, so there is still no "proposed but not yet proven" intermediate state to reason about — there is still no "proposed but not yet proven" intermediate state, no sealing deadline / cancellation horizon / cancellation cascade / timing-exemption net (the v15 complexity reduction is retained). But what "verification passed" now yields is the current best candidate (provisional) of that window, rather than immediate finality; finality is granted at settlement-window close to the heaviest proven candidate in the window (§5.6). Under normal conditions, with no competition, the aggregator's candidate is the only candidate and close is finality — the semantic difference appears only under attack. **Withdrawal and L2→L1 bridging effects execute only from window-final state**, never from provisional state.
- Batch size is bounded above by proving cost, and by how much body data can be published to L1 before the candidate lands; it is no longer bounded by what one transaction can carry. Batches need not align with epoch boundaries.
- **Block bodies are published separately from candidacy, and a candidate references them** : this is what keeps the landing transaction's size independent of how many blocks it claims. Bodies are carried to L1 by ordinary blob transactions through a $\mathrm{postData}$ entry point, which reads the versioned hashes of the attached blobs and records them in an append-only registry together with the L1 height at which they arrived. A candidate then names, in its proof's public inputs, the ordered list of versioned hashes its segment consumes; the L1 entry point checks that each is registered, and the circuit proves that the referenced blob contents decode to exactly the bodies of the blocks $[\mathrm{m}{+}1..\mathrm{k}]$ whose headers it has verified. Binding content to commitment can be done inside the circuit or discharged on L1 through the point-evaluation precompile against an evaluation the circuit exposes as a public input; which is cheaper at the segment sizes this protocol needs is a calibration item (§11), not a question of soundness. **Timing is free**: data may be posted at any point before the candidate that consumes it lands, so the design does not require bodies to be published as blocks are produced. Posting data mid-window does not disturb the §5.6 baseline freeze, and for the same reason the message queue does not: the registry is append-only and entries are named by versioned hash, so an entry's content is immutable and the moment it arrives cannot change the verdict on any candidate — a candidate either references a registered hash or it does not. What the freeze covers is the cursors and the state, not the contents of append-only L1 structures (property P12 of Appendix C).
- **Two consequences, and one non-consequence** . The throughput ceiling is gone: the landing transaction carries a proof, a reference list and the end-state tuple, so the chain's advancement per settlement window is bounded by L1's blob bandwidth over the window rather than by what fits in one transaction — with 150 L1 slots per window that is roughly two orders of magnitude more room than a single transaction affords. **Data is also no**

longer paid for twice: because references are permissionless, a later candidate in the same window reuses the blobs an earlier one already posted and pays only for its own increment, which is what makes a fallback lander's marginal cost small when it extends an aggregator's published prefix. The non-consequence is equally important: **this does not give the unlanded tail protocol-level availability**. Data reaches L1 only when someone intends to land it, so the deep-reorg residual of §5.4 and the preconfirmation DA assumption of §5.5 are unchanged. Publishing bodies continuously as blocks are produced is available as a strengthening that would close both, at the cost of paying for data that may be orphaned; it is a separate decision and is not assumed here.

- **Referencing another party's data is permitted; reusing another party's proof is not** . These are separable and must stay so. A blob reference is a versioned hash naming public bytes, and anyone may name it — that reuse is the point. A proof is bound to its beneficiary through the public inputs, per the anti-plagiarism clause of §5.6, so lifting a competitor's proof from the mempool and resubmitting it under a different beneficiary remains impossible. The party that posts data is not thereby granted any claim over candidates that reference it.
- **Future slots must not be landed** : the slot of a batch's last block must be $\leq$ the $\mathrm{L2\_slot}$ corresponding to the timestamp of the L1 block in which it lands (the permitted clock skew is initially 0) — checked at the L1 entry point and proven identically in the circuit. The lookahead is published at least $\mathrm{H\_look}$ = 768 slots in advance; without this constraint a builder could pre-sign blocks for future slots, and a batch could push the landed chain head ahead of wall-clock time — destroying the timestamp/anchoring/forced-expiry semantics and artificially driving the lag of §6.3 negative. Correspondingly, the arithmetic definition of lag is truncated below at 0 (safe arithmetic).

6.2 Cadence

- Target landing cadence: the aggregator should keep the lag lag defined in §6.3 (the number of slots by which the tip of the best provisional candidate trails L1 wall-clock time — observable on L1, referring to no P2P state; lag is measured on the provisional candidate, so that the liveness determination is not dragged down by window delay) within Δ_lag. A proving latency of about 10–15 minutes means that in steady state provisional landing trails the live chain head by about 15–20 minutes; window-final finality adds one further $\mathrm{W\_settle}$ — the decoupling of the preconfirmation experience (seconds-scale) from the finality cadence (minutes-scale) is unchanged, and it is the former that users perceive. The ladder of four levels of certainty, and the two lags:

- L2→L1 bridge (withdrawal) delay = landing cadence + $\mathrm{W\_settle}$ (withdrawals execute only from window-final state, §6.1).
- **No fast close (fast path)**: even when a window contains only a single candidate, the full $\mathrm{W\_settle}$ is awaited before finality — a test for "closing early" would reintroduce gameable timing, and v1 does not do it; it is listed in §11 as an optimization (for example, "a candidate covering all known signature-chain heads may close early" may be introduced only after it has been proven not to be manipulable by block withholding).
- Two lags, each with its own role ($\mathrm{lag\_final}$ = the amount by which the window-final head trails — used for safety/revocable- exposure accounting and for fallback accounting (the

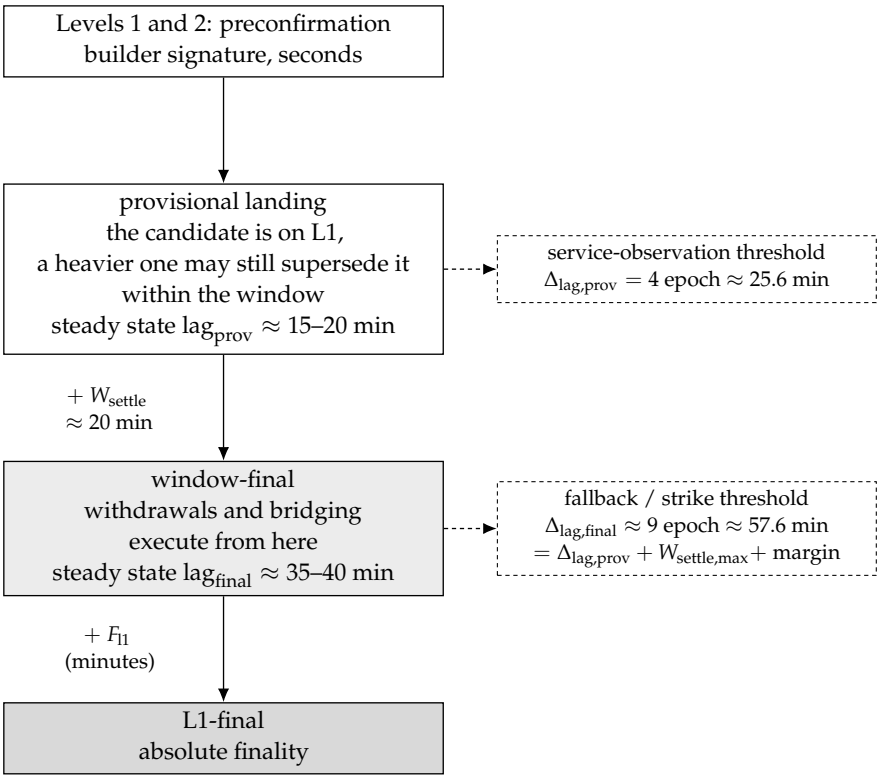

Figure 6: The four-level certainty ladder and the two lags: preconfirmation is a matter of seconds, and provisional landing is one settlement window away from window-final. lag_{prov} serves only to observe the service tempo, while it is $\text{lag}_{\text{final}}$ that enters fallback and slashing accounting.

next item and §6.3). Two thresholds, in one-to-one correspondence (splitting only the adjudicated quantity without splitting the threshold would leave the fallback window permanently open): $\Delta\_\mathrm{lag,prov}$ (= the old Δ_lag, initially 4 epoch ≈ 25.6 min, setter invariant $\geq$ the upper bound of the normal provisional lag band) applies to $\mathrm{lag\_prov}$; $\Delta\_\mathrm{lag,final} = \Delta\_\mathrm{lag,prov} + \mathrm{W\_settle\_max(wall\text{-}clock\ conversion)} + \mathrm{margin}$ (initially ≈ 9 epoch ≈ 57.6 min.2 min, but $\Delta\_\mathrm{lag,prov} + \mathrm{W\_settle\_max}$ = 128 + 150 L1 slot = 55.6 min already exceeds it, so 8 epoch violates the very formula on this same line; once model property P9a was made non-vacuous (deployment values declared independently, inequalities cross-checked) this was caught immediately, and the value was changed to 9 epoch = 288 L1 slot, with a margin of 2 min) applies to $\mathrm{lag\_final}$. Rationale: under window finality the steady-state $\mathrm{lag\_final} \approx \mathrm{lag\_prov} + \mathrm{W\_settle} \approx 35\text{–}40\ \mathrm{min}$, which by itself already exceeds the old threshold of 25.6 min — if fallback/strikes were adjudicated on $\mathrm{lag\_final}$ while the old threshold were retained, the fallback window would be permanently open in a completely healthy system, the "cost + markup" reward commitment would remain in force in perpetuity, and a diligent aggregator would accrue strikes continuously until termination; the current rule the adjudicated quantity without raising the threshold is precisely this bug. The honest bound on revocable exposure: although a provisional candidate has already landed on L1, it can still be superseded before close, so the worst-case revocable set of tier-2 preconfirmation = the unlanded tail + the provisional increment within the window, and the true bound = $\Delta\_\mathrm{lag,prov}$ + $\mathrm{W\_settle}$ (wall clock) + fallback response ≈ $\Delta\_\mathrm{lag,final}$ + fallback response — it can no longer be written as "≈ Δ_lag" on its own; elsewhere in this document, the Δ_lag appearing in depth bounds of the form "depth ≈ Δ_lag + fallback response" is, per to be read as the fallback-window-opening threshold $\Delta\_\mathrm{lag,final}$ (a new name for the same quantity; the bound is unchanged, only the naming is made honest). In order to pin this bound down, $\mathrm{W\_settle}$ is given, besides its lower bound, a consensus upper bound $\mathrm{W\_settle\_max}$ (§11; including the margin for wall-clock conversion when L1 slots are missed), which is folded into the user-facing semantics and the wording of §5.4.

6.3 Landing Obligation and Permissionless Fallback (This Is the Aggregator's Only Obligation)

- **Lag (L1-observable)**: $\mathrm{lag\_*} = \mathrm{L2\ slot\ number\ of\ the\ current\ L1\ timestamp} - \mathrm{slot\ number\ of\ the\ reference\ head}$; taking the tip of the best provisional candidate as the reference head yields $\mathrm{lag\_prov}$, and taking the window-final head yields $\mathrm{lag\_final}$ (the §6.2 split). Terminological convention: in the remainder of this section (window opening / strikes / late fees / threshold-riding determinations), bare $\mathrm{lag}/\Delta\_\mathrm{lag}$ are always to be read as $\mathrm{lag\_final}/\Delta\_\mathrm{lag,final}$; $\mathrm{lag\_prov}/\ \Delta\_\mathrm{lag,prov}$ carry only the service-level observation of §6.2 and enter no penalty rule; the Δ_lag appearing in historical calibration notes refers to the single threshold of that time, whose value is now inherited by $\Delta\_\mathrm{lag,prov}$. Normative definition: $\mathrm{L2\_slot(t)} = \mathrm{floor((t - GENESIS\_L2)\ /\ 1\ second)}$, where t is the timestamp of the L1 execution block in which the determination is made (constrained by L1 consensus, a single proposer has only second-scale jitter influence); it references no L2 or P2P data. Both quantities are pure functions of L1 state, and any contract call can compute them.
- The causal chain of the fallback window and the strikes (the precise definition of each link is given in the subsequent items of this section):

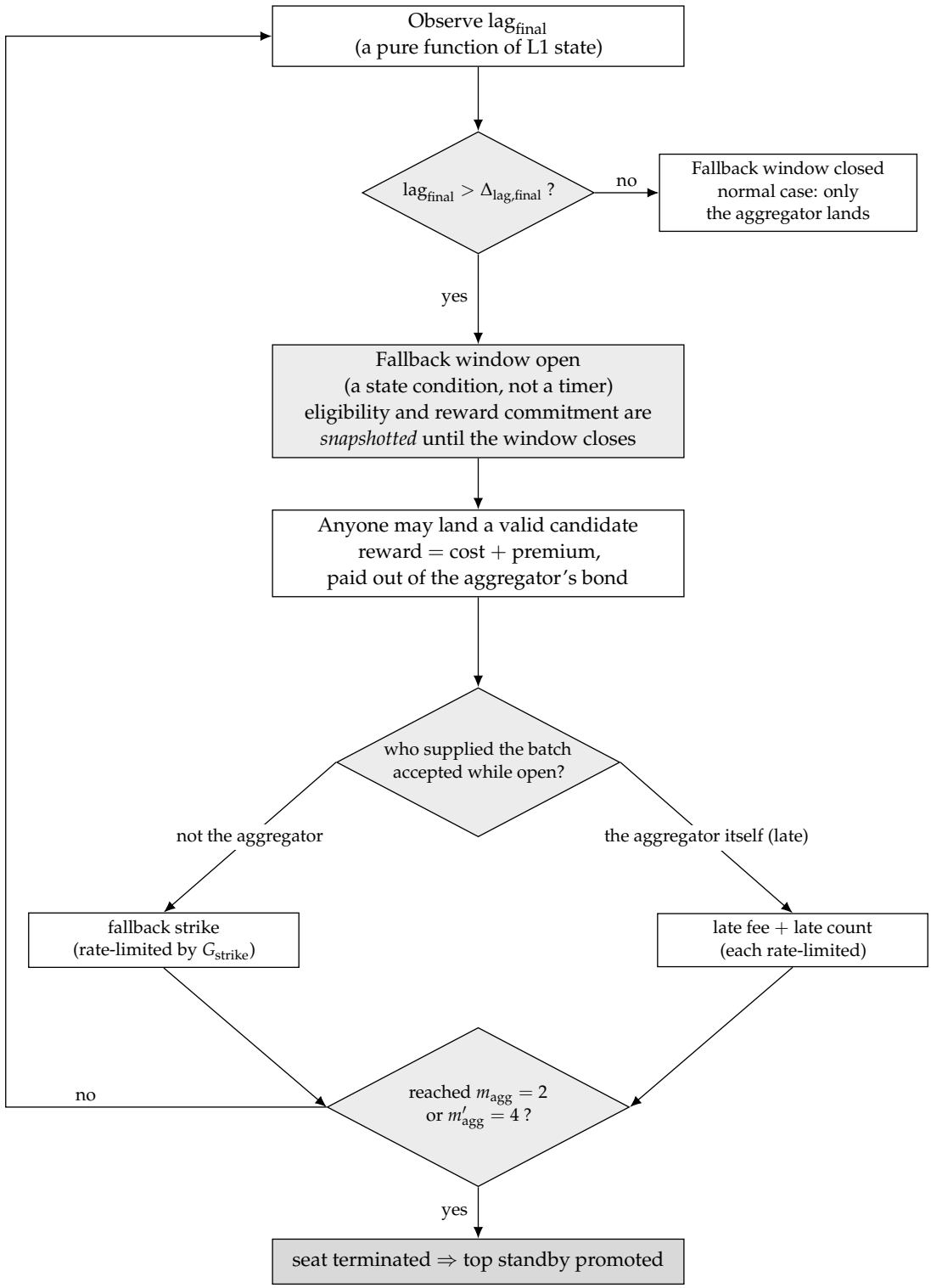

Figure 7: The causal chain of the permissionless fallback window and of seat strikes: fallback eligibility is opened by a state condition, namely $\text{lag}_{\text{final}}$ crossing its threshold as observed on L1. Who lands while the window is open decides what is recorded: a fallback strike, accrued at a rate limited by G_{strike}, or a late fee priced on the overshoot together with a separately rate-limited late count. Strikes accumulating to the threshold terminate the seat.

- **The fallback window is a state condition, not a timer**: whenever $\mathrm{lag\_final} > \Delta\_\mathrm{lag,final}$ (initially $\approx$ 9 epochs $\approx$ 57.6 minutes = $\Delta\_\mathrm{lag,prov} + \mathrm{W\_settle\_max} + \mathrm{margin}$; for the derivation see §6.2; $\Delta\_\mathrm{lag,prov}$ is initially 4 epochs $\approx$ 25.6 minutes), the fallback window is automatically in the open state — anyone may land any valid batch and be paid from the aggregator's bond at indexed cost (L1 fees + proving cost + markup) (a transplant of the fault-pays model of v15 §6.7). The window does not close because "the aggregator landed a batch" — it closes only because $\mathrm{lag\_final}$ falls back within $\Delta\_\mathrm{lag,final}$. Micro-batches are therefore ineffective: landing 1 block does not reduce the lag, and the window stays open regardless. Fallback eligibility and reward are determined by $\mathrm{lag\_final}$ and are snapshot-held until the settlement window closes (if the determination were made on the provisional lag, a malicious accomplice could first land a short candidate that zeroes out lag_prov and thereby switch off fallback eligibility before an honest heavier candidate lands, cutting off its proving-fee compensation; rational fallback landers then withdraw and the bad candidate closes the window unopposed): once the fallback window has opened under $\mathrm{lag\_final} > \Delta\_\mathrm{lag,final}$, the eligibility and the "cost + markup" reward commitment remain valid for every candidate within this settlement window until close; the reward is paid to the closing winner and to every candidate that strictly improved the best chain — an honest improver overtaken by a heavier candidate does not work for nothing, and a short provisional candidate cannot take the eligibility away. **Fixed cost is metered per** $(\mathrm{count},\ \mathrm{tip\_slot})$ **level, not per strict improvement.** Call a *level* a distinct $(\mathrm{count},\ \mathrm{tip\_slot})$ pair that an accepted candidate has reached during the window. Each level carries exactly one $\mathrm{C\_fixed}$, paid at close to that level's *holder* — the submitter of the accepted candidate with the smallest $\mathrm{tip\_hash}$ at that level. On top of that, φ_adv per block added beyond the previous best, and φ_data per byte of body data the claimant published and the winning chain consumed (§6.1); without that third term the party whose landing the protocol depends on when an aggregator withholds data would be paid for its proof but not for the blobs it had to post. $\mathrm{C\_fixed}$ **is set at or just below the true indexed cost of one proof plus one inclusion**, which is what keeps prefix-splitting from paying.

  Why per level and not per improvement: because the tie-break is reachable only through equivocation (§5.2) *and* slashing is idempotent (§4.1), a builder scheduled at one slot can sign arbitrarily many headers for it, submit them in decreasing $\mathrm{tip\_hash}$ order, and have every one of them count as a strict improvement — while being slashed exactly once. Reimbursing per improvement would hand that builder one $\mathrm{C\_fixed}$ per candidate for a single $\mathrm{L\_eq}$, draining the aggregator's bond to $\mathrm{R\_window\_max}$, forcing seat termination and haircutting honest claims once the rail binds. Level metering collapses the whole grind to the single $\mathrm{C\_fixed}$ its one level carries, so the attacker pays $\mathrm{L\_eq}$ plus one proof per attempt and recovers one. The honest case is unharmed, and the incentive points the right way: a party that displaces a malicious same-level candidate *becomes* the holder and takes that level's $\mathrm{C\_fixed}$, while the party it displaced keeps only the marginal-block reward it actually earned. Two properties follow, and both are needed. *Nobody works for nothing*: a candidate that improves without adding blocks still recovers $\mathrm{C\_fixed}$ in full — a later $\mathrm{tip\_slot}$ opens a new level, and a winning $\mathrm{tip\_hash}$ makes the submitter the holder of its level — because the settlement order is the whole triple of §5.2 and not the block count alone. This must hold for *each* improver independently: permissionless fallback is the only liveness backstop when the aggregator fails, and a rule under which honest improvers land at a loss does not have a backstop at all. *Splitting earns nothing*: the φ_adv terms are metered on marginal blocks and therefore telescope, so dividing one advance of A blocks into k

candidates pays exactly the same $\varphi\_\mathrm{adv} \times A$ as landing it once; the only component that scales with k is the fixed-cost reimbursement — each prefix does open a genuine new level — and since $\mathrm{C\_fixed}$ does not exceed what the grinder actually spends per prefix, splitting is at best break-even and never profitable. A hard ceiling $\mathrm{R\_window\_max}$ on total bond outflow per window remains, with pro-rata distribution above it, but it is a safety rail rather than a metering rule: it is set far above a plausible honest window, and its binding at all is an alarm condition rather than normal operation.

Two superseded formulations, recorded so neither is reintroduced. The first capped the window total at a single $\mathrm{C\_fixed}$ allowance while every improver claimed one, so the pro-rata factor fell below 1 for every $k > 1$ and *every* improver — including the one that did the work — was paid below its own fixed cost, at 0.46 for three improvers and 0.30 for five. The second paid one $\mathrm{C\_fixed}$ per strict improvement, which is the bond-drain above. Both survived review because the property test asserted only that an improver was paid something. Appendix C now asserts cost recovery and level metering, and pins both superseded rules against the same inputs so neither regression can return silently. Reimbursing each strictly-improving candidate for its *full* proving and inclusion cost would be gameable: a party holding one valid N-block tail could submit the prefixes of length $1, 2, \ldots, N$, each of which strictly improves the block count, and collect N full reimbursements plus N markups for a single tail's worth of real advancement — draining the liveness bond and potentially terminating the seat during exactly the degraded period the bond exists to cover. The "every candidate needs a real proof" anti-spam argument does not deter this, because that very proving cost is what is being refunded; and the requirement that proofs support incremental continuation from a proven prefix (item 6 of §11, needed to defang drip-feed stalling) makes the attack cheaper still, since the grinder's marginal cost per prefix is incremental while a full-cost claim is not. Pricing $\mathrm{C\_fixed}$ at cost removes the incentive without weakening the honest-improver guarantee: splitting an advance into more candidates changes only the submitter's own gas bill. Note that a per-block reward alone would not have sufficed. Because §5.2 orders candidates by the triple $(\mathrm{count},\ \mathrm{tip\_slot},\ \mathrm{tip\_hash})$, a candidate can be strictly better while adding no blocks at all, and paying such a candidate nothing would contradict the guarantee above and would leave a malicious same-count candidate unopposed because displacing it costs a proof and earns nothing. Nor does paying it open a griefing vector, but the reason is level metering rather than the price of equivocation: an attacker can manufacture unboundedly many tip-only candidates for one $\mathrm{L\_eq}$, and what denies it a payout is that they all share a single level and therefore a single $\mathrm{C\_fixed}$. The residual exposure is bounded on the other side as well — the fallback window is open only while $\mathrm{lag\_final} > \Delta\_\mathrm{lag,final}$, that is, only while the aggregator is already in breach, which is exactly the circumstance the bond exists to fund. $\mathrm{C\_fixed}$ (subject to the at-cost constraint above), φ_adv, $\mathrm{R\_window\_max}$ and the treatment of a window whose winner adds no blocks over its baseline are calibration items under §11.

- **Fault and incriminating evidence (mechanically decided on L1)**: one "strike" = a batch from someone other than the aggregator was accepted while the fallback window was open. The elegance of this definition lies in its zero false positives: if there is simply no block available to land on P2P (all builders absent), then nobody can land anything and the aggregator is not struck for "having nothing to land"; whereas once someone does land, that very fact proves on L1 that "there was landable content and the aggregator did not land it" — the dereliction is self-evidencing by fact, with no need to observe P2P. Each

strike slashes one tier of the aggregator's liveness bond. Strikes accumulate with the duration for which the window stays open and are rate-limited by $\mathrm{G\_strike}$: the unit of a strike is changed to a sustained open-window interval — for each full $\mathrm{G\_strike}$ (initially 1 epoch) that an open-window period (a contiguous interval in which $\mathrm{lag} > \Delta\_\mathrm{lag}$) persists, one strike is counted if ≥ 1 non-aggregator batch was accepted within that interval. Two properties are preserved simultaneously: (i) rate limiting against spam: however many micro-batches an attacker splits its landings into within the same $\mathrm{G\_strike}$ interval, only one strike is counted, and the attacker cannot accelerate time; (ii) sustained dereliction necessarily accumulates: as long as the window stays open and fallback landings keep occurring, the strike count rises once per $\mathrm{G\_strike}$, so $\mathrm{m\_agg}$ (2 strikes) is reached within ≈ 2 epochs and the aggregator is replaced — drip-feed stalling falls precisely into this branch. The first strike still requires the window to have been continuously open for $\geq \mathrm{G\_strike}$ (the grace period for transient faults is unchanged). The symmetric reverse weaponization (a malicious fallback lander drip-feeds micro-batches to invalidate the aggregator's catch-up proofs and to run up strikes on its behalf) is met as follows: the aggregator's own batches never incur a strike and it may bid arbitrarily high to get in first, and batch proofs must support incremental continuation/aggregation from an already-proved prefix (an engineering requirement, folded into §11 item 6) — once a micro-batch that advances the chain head no longer forces anyone to re-prove from scratch, the drip-feed's teeth are pulled; the residual risk is still "sustained L1-level censorship of the aggregator's landing transactions" (the most expensive censorship category, already priced in v15 §11.5), and is explicitly accepted. Bond exhaustion is itself termination: late fees and strikes draining the liveness bond through the floor (below the minimum maintenance amount, a calibration item) counts as a termination trigger, so the fees are also an independent termination path and no longer mere bleeding.

- **A late landing is itself charged — closing the "threshold-riding" strategy** : counting strikes solely on "a fallback batch was accepted" has one hole: an online malicious aggregator can squat right at the threshold — as soon as lag just exceeds Δ_lag and the fallback window opens, it front-runs the fallback lander with its own catch-up batch, closes the window, never takes a strike, keeps finality permanently riding the Δ_lag line, and burns the fallback lander's proving cost for nothing. The fix (equally mechanically decidable on L1): any batch accepted while $\mathrm{lag} > \Delta\_\mathrm{lag}$ — including the aggregator's own — charges the aggregator a late fee priced by the excess (deducted from the liveness bond), and the event of the aggregator's own late batch being accepted is recorded in a lighter violation counter, which upon reaching $\mathrm{m\_agg'}$ (initially 4) likewise triggers termination. The counter is attached to "a late batch was accepted" rather than to "the window was opened": the window may open for reasons unrelated to the aggregator, such as all builders being absent or a proving-system failure, and in that case nobody can land at all, so any count would be unjust; whereas "a late batch was accepted" (whether the aggregator's own or a fallback lander's) itself proves on L1 that "there was landable content", so zero false positives holds. The threshold-riding front-running loop remains closed: every time the front-runner closes the window it necessarily lands a late batch of its own, and every such time is counted. Honest degradation: "not a penny is charged during an externality-caused failure" currently holds fully only for the fallback strike; the late fee and the late counter can still hit "the honest aggregator that lands the first recovery batch after a network-wide proving-system failure is resolved" (such a recovery batch is necessarily late). Fix one: the late counter accumulates under the same $\mathrm{G\_strike}$ rate limit as the fallback strike (for

each G_strike of sustained window opening within whose interval one of the aggregator's own late batches was accepted, at most one is counted; otherwise an aggregator riding the threshold with its own micro-batches could likewise pin m_agg' at 1). Fix two: the exemption mechanism for systemic proving outages is elevated to a blocking open item (§11 item 8) — until it is fully defined and mechanically verifiable, the "zero false positives" claim is limited to the fallback strike and does not cover the late fee. The landing gas lost by a front-run fallback lander is compensated out of the late fee according to a pre-registered intent (the detailed rules are folded into §11 item 3).

- **Closing the zero-penalty reset of "self-wiping the lag + landing a worse chain"**: charging solely on "whether lag is $> \Delta\_lag$ at acceptance time" gives the aggregator a threshold-pinning loop: it can time its landing exactly at $lag = \Delta\_lag$ (not $>$) and land a short chain of its own that overrides the healthy tail; acceptance then carries a zero late fee, the lag drops back after landing, and the cycle repeats every proving period — finality is suppressed indefinitely at zero penalty and the seat is never replaced. Two closures: (a) **Landing a worse chain is self-defeating (the main closure)**: each worse short chain it lands is merely one candidate inside the window — anyone can land that healthy content tail with a proof into the same window, and by the total order it directly overrides the malicious candidate; the malicious candidate wastes gas + proving fees and suppresses nothing. "Zero penalty, repeatable" becomes "zero revenue, guaranteed loss", and no evidence submission or slashing adjudication is required. (b) **The lag debt is settled against the pre-acceptance state, leaving no room for integer threshold-riding**: the determination for the late fee and the late counter is changed to $lag \geq \Delta\_lag$, and the fee is priced as $late\_units = \max(0, preLag - (\Delta\_lag - 1))$ — that is, at $lag = \Delta\_lag$ we have $late\_units = 1$ (the minimum non-zero late fee, no longer an "excess = 0 exemption point"); the m_agg' counter is changed to maintain independently, for "one of the aggregator's own late batches was accepted", a rate-limit state keyed on the L1 slot of the last count (no longer depending on the old condition "the window has been continuously open for a full G_strike", which is undefined at the equality point). Timing a landing precisely at $= \Delta\_lag$ is thereby no longer penalty-free and no longer strike-free. Rate-limit units are unified : the normative unit of G_strike is L2 seconds (1 epoch = 384 L2 seconds); the m_agg' rate-limit state stores "the L1 slot of the last count", and the comparison converts via $(\text{current L1 slot} - \text{last L1 slot}) \times 12 \geq G\_strike\_L2s$ — eliminating the ambiguity in which "a count of L1 slots is used directly as L2 epochs", which causes a factor-of-12 discrepancy. The two act together: the self-wiping-lag loop either lands the genuinely better tail (harmless) or is overridden inside the window at a pure loss plus a late fee, and the "Byzantine aggregator is bounded" claim of §6.3/§8 is restored (under the same fallback-feasibility condition that someone within the window is willing to land the honest tail).
- **Termination and replacement**: an accumulated m_agg fallback strikes (initially 2) or m_agg' of the aggregator's own late batches (initially 4) $\rightarrow$ the seat is terminated and the highest standby is promoted, with a promotion delay of 1 epoch (in this design the aggregator does not issue preconfirmations, so a replacement is not perceptible at the service level and the delay can be aggressive).
- **Reward anti-grinding**: the aggregator's service reward is charged by the number of slots that a landing advances, not by the number of batches — a micro-batch not only fails to suppress the fallback window, it also earns nothing.
- **Aggregator liveness is a "bound under an economic assumption", not a pure protocol**

bound : strikes and $\mathrm{m\_agg}$' accumulate only when a batch is accepted inside the fallback window. If the aggregator silently stops landing and nobody performs a fallback landing — because the proving cost exceeds the reward, because of an L1 fee spike, or because no prover is available at all — then no strike is produced, the seat does not change hands, and finality may stall indefinitely. The bound on the "Byzantine aggregator is bounded" row of the §8 table is therefore conditional: it holds if and only if permissionless fallback is economically feasible (someone is willing and able to bear the proving and gas cost of the fallback). This is an economic/engineering assumption, not protocol-enforced liveness; stating it explicitly is an honesty clause of the same kind as §1's "honest majority is an economic assumption". The indexed-cost cap on the fallback reward and the fallback incentive under extreme congestion are listed as calibration items in §11 items 3 and 7, and the availability/cost of fallback provers is formally folded into the §1 threat model and §11.

- **The availability precondition for fallback (this item must be stated explicitly)**: a fallback landing requires the block bodies in hand, and before landing, block-body availability is only best-effort guaranteed by P2P (§5.5). The exact condition for "the Byzantine aggregator is bounded" is therefore: the unlanded tail, or some available fork of it, is available to at least one lander willing to perform a fallback. By the structural self-healing of §5.5, this reduces to "there exists ≥ 1 non-colluding builder in the lookahead" (which automatically routes around the segment whose bodies are withheld and produces a fork that anyone can land) — that precondition is directly implied by the §1 trust model (an honest majority of builders) with ample margin, so within the model the boundedness of a Byzantine aggregator is unconditional. If the builders and the aggregator collude as a whole (bodies kept private, nobody able to land anything), the strike mechanism indeed does not fire — which is the correct behavior of the zero-false-positive principle (the stall is then attributable to the cartel, not to the individual aggregator); by the §1 trust model that case is out-of-model, the protocol offers it no finality guarantee, and With no forced inclusion, there is no defense in depth either — the former route "a user submits a forced entry $\rightarrow$ the deadline passes and the lag exceeds its limit $\rightarrow$ anyone advances finality with a forced-only block" no longer exists.
- **Key property**: because landing requires no authority (§0 item 3), the fallback requires no new trust. Aggregator offline: the sequencing service is entirely unaffected (builders keep producing blocks and preconfirmations keep being issued), and finality is maintained near Δ_lag by fallback landers. Aggregator online but maliciously stalling (Byzantine): micro-batches cannot save it — once the lag exceeds the limit the fallback opens, once a fallback landing occurs a strike is counted, and after $\mathrm{m\_agg}$ strikes the aggregator is replaced; under the availability precondition of the previous item, the worst-case finality lag is pinned at Δ_lag + the fallback response time, and is bounded. These two cases are listed separately from the full-cartel case in the table of §8.

6.4 Stall and Gap Recovery — Ordinary Block Production Takes Over in One Hop via the Landing Head, with No Episode

After a stall of hours or even days, recovery is merely "one landing + one tier (ii) block":

- **The mechanism (just one rule)**: when the gap > $\mathrm{G\_max}$ (consecutive absences exceed the cap), or when the aggregator withholds landing so that normal block production can-

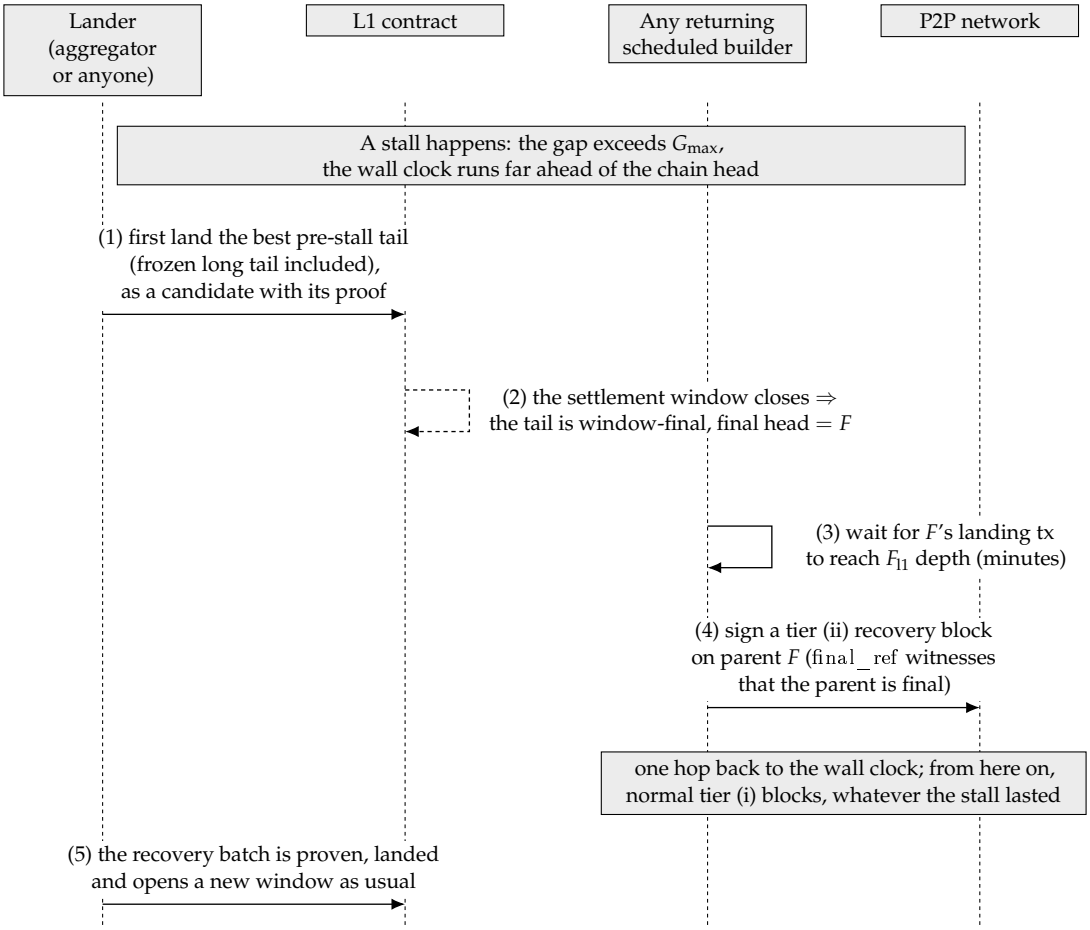

Figure 8: Recovery after a stall of hours or even days: first land the best pre-stall tail and let it close into the final head F, then let any returning builder sign a tier (ii) recovery block on parent F, which reattaches the ordering to the wall clock in a single hop. Recovery involves no separate sub-protocol.

not continue on the unlanded tail — any lookahead builder of the current turn takes the current final L1 landing head as parent and produces an ordinary block under tier (ii) of §4.2 (a content block). That block's slot = the wall-clock slot of its turn, and the parent distance is arbitrary (tier (ii) has no cap). L2 sequencing thereby recovers: the landing head is reconnected to near the wall clock, and subsequent builders produce blocks on top of it normally (tier (i)). The production/landing of the recovery block itself has no announcement, deposit, freshness window, or state machine — it is merely an ordinary candidate in the §5.6 settlement window, competing with the other candidates by the total order. The ordering rule of landing the best chain first is given in the next item: if a frozen long tail exists from before the stall, land it first and recover from its tip, so that its preconfirmations are not lost.

- **Land the best (including frozen) chain first, then recover from its final tip — do not discard the tail** : The correct order is: (1) by the total order of §5.2 the pre-stall long tail is still the best chain ("cannot be extended over P2P" $\neq$ "should be discarded"), so a lander lands it first (honoring all of its preconfirmations; the $\mathrm{G\_max}$ gap merely prevents P2P from extending it further and does not prevent proving and posting it); (2) once it has landed and reached $\mathrm{F\_l1}$ it becomes the new final tip, and the recovery block is produced with that new tip as parent under tier (ii) of §4.2, reconnecting to the wall clock in one hop from there. The "recovery vs. fork-choice deadlock" thus disappears (not by discarding the tail, but by landing the tail first), and the preconfirmations of the healthy long tail are not sacrificed. Only when there genuinely is no longer/better chain (a true total stop, with not even a frozen tail) is the recovery block itself the best chain and landed directly. Skipping a better tail in order to land a recovery block is self-defeating: anyone can land the better tail into the same §5.6 window and it wins.
- **"One hop" means re-rooting; sustained progress still requires the normal liveness floor**: the recovery block R (tier (ii)) re-roots sequencing onto the landing head in one hop; but R is itself an unlanded P2P block, and to genuinely reconnect to the wall clock a subsequent builder must extend it within $\mathrm{G\_max}$ (tier (i), exactly as for every normal block) — proving and landing R takes 10–15 minutes, and during that time sequencing continues over P2P on top of R (the normal "preconfirmation in seconds, finality in minutes" rhythm, not a new problem). "A recovery block never expires" refers precisely to the landing validity of R (tier (ii) has no gap cap, so R can land whenever it is proved), not to R being extendable without maintenance. The exact condition for recovery liveness is therefore not "one builder produces one block" but "lookahead builders keep returning at a density of ≥ 1 per $\mathrm{G\_max}$ window" — that is, the $\mathrm{G\_max}$ liveness floor that normal block production already requires. If only a single builder returns and its lookahead turns are sparser than once per $\mathrm{G\_max}$: nobody extends R within $\mathrm{G\_max}$ $\rightarrow$ R itself freezes (§5.2) $\rightarrow$ the next builder produces another $\mathrm{R'}$ on the same old landing head (R is unlanded and not final, so it cannot be referenced by tier (ii)). This retry loop is benign and loses no ground (the landing head has not moved, and each attempt re-roots from the same point); once participation returns above the $\mathrm{G\_max}$ floor, some R is extended, lands, and recovery completes. This floor is identical to the normal block-production floor and is not a new requirement specific to recovery; a genuine long-term outage of all participants is a matter for social recovery and is out-of-model. (the $\mathrm{G\_max}$ density floor can only be met by scheduled builders.)
- **Why a stall of any duration can be recovered in one hop (this is the fundamental im-**

provement over the old design): tier (ii) imposes no cap in the gap dimension, so a recovery block does not expire because its parent distance is too large (there is no "expiry" in the gap dimension). Whether the stall lasted 2 hours or 2 days, recovery is always "produce one block with the landing head as parent → one round of proving → landing". The whole family of problems from the old design in which "proving latency blows through the gap freshness window" therefore disappears. Note the distinction of dimensions: what is stated here is that the gap does not expire; the other dimension, anchor freshness, still applies (see the next item + §7) — "never expires" refers precisely to the former, not the latter, and the two are not in conflict.

- **The precise boundary of "one hop" — the transient** $\mathrm{F\_l1}$ **wait** : the parent of a tier (ii) block must be a final landing head, whereas §5.6 lets only candidates that extend the current canonical landing tip land. The two fail to coincide in exactly one transient: the stall begins shortly after a landing and the current tip has not yet reached $\mathrm{F\_l1}$. In that case the recovery block can only wait for the current tip to become final (an extra $\leq \mathrm{F\_l1}$, on the order of minutes) — building on an older final head would fork off the newer tip and §5.6 would refuse the landing. The precise recovery bound is therefore max(0, $\mathrm{F\_l1}$ + $\mathrm{D\_anchor}$ − the time the current tip has already existed) + one round of proving and landing (the earliest signable point = parent landing + $\mathrm{F\_l1}$ + $\mathrm{D\_anchor}$): a steady-state disaster stall (in which the last landing is long past that point) degenerates to a pure one hop, while the worst transient, a stall starting immediately after a landing, adds a wait of $\leq \max(\mathrm{W\_settle}, \mathrm{F\_l1})$ + $\mathrm{D\_anchor}$ (≈ 26.4 minutes at the initial values, on the order of minutes — the parent must be both window-final and L1-final, so the two waits overlap rather than add, and §4.2 states the same bound). In either case this is independent of the total stall duration (it does not grow with 2 hours or 2 days).
- **The constraint anchor freshness places on the recovery block — "never expires" needs a qualifier** : what tier (ii) eliminates is gap expiry (no gap cap, so R is valid however far its parent is), but anchor freshness (§7, $\mathrm{D\_anchor\_max}$) still applies: R pins its anchor at signing time and must land within $\mathrm{landing\ L1\ block\ height} - \mathrm{anchor} \leq \mathrm{D\_anchor\_max}$, failing which R's anchor is too stale, does not pass §7, and must be re-signed and re-proved against a fresh anchor. The precise statement is therefore: "a recovery block never expires because of the gap" holds, but it must be proved and landed within the $\mathrm{D\_anchor\_max}$ freshness window of its anchor. The key difference from the old re-anchor s_ra: the old $\mathrm{s\_ra}$ freshness window was structurally pinned to $\mathrm{G\_max}$ (a 64-second wall-clock window), which proving latency (10–15 minutes) necessarily blows through, whereas $\mathrm{D\_anchor\_max}$ is a freely settable parameter, and its setter invariant (the full formula is in §11) guarantees that a recovery block does not expire under normal L1 liveness — the old design lost because its window was structurally too small, not because of the mechanism itself. Since The stale-anchor tail deadlock is resolved by the window geometry of §7 (any tail a fallback lander is authorized to land necessarily still has a fresh anchor). If L1 censors R's landing transaction continuously for longer than $\mathrm{D\_anchor\_max}$ and R has not been committed, R is rebuilt and re-proved with a fresh anchor (with no loss of sequencing) — that case is a degradation of L1 liveness and does not occur under the §1 assumption that L1 is safe and live.
- **Determinism and benign stranding** : the parent of a tier (ii) block = a final landing block (having reached the $\mathrm{F\_l1}$ finality depth, so the predicate is stable under shallow L1 reorgs; linked to the L1-reorg item of §11); determinism at landing time is backstopped by the §5.6

landing rule — a batch may only extend the current canonical landing tip. If a recovery block built on an old tip finds that the tip has already been advanced → benign stranding, and it need only be rebuilt (there is no episode or deposit that can go wrong). And the tip being advanced ⇔ a new landing occurred ⇔ the chain has not truly stopped; during a genuine stall the tip does not move and the recovery block necessarily lands.

- **L1-sync backlog catches up under a per-block cap** : "one-hop recovery" refers precisely to L2 sequencing recovering in one hop; the L1→L2 message backlog accumulated during a long stall catches up under a per-block cap: the L1→L2 message-processing cursor $\mathrm{m\_consumed}$ advances by $\leq \mathrm{C\_anchor}$ messages per block (§7 — $\mathrm{C\_anchor}$ is bound to the message cursor; the anchor reference itself has no per-block advancement cap and may jump to a fresh height in one hop; the earlier text of this line, "the anchor's L1 advancement is $\leq$ C_anchor per block", was a leftover in conflict with §7 and has been corrected). The full accounting is therefore: sequencing = 1 hop; L1-sync backlog = $\mathrm{backlog\ /\ per\text{-}block\ cap}$ blocks (after recovery, at 1 block per second, this is usually cleared in seconds to minutes).
- **The exact condition for recovery liveness** : a tier (ii) content recovery block still requires $\mathrm{signer} = \mathrm{lookahead(slot)}$, so the exact precondition for discretionary recovery = lookahead builders keep returning at a density of ≥ 1 per $\mathrm{G\_max}$ window (= the $\mathrm{G\_max}$ liveness floor of normal block production, not something specific to recovery; a single sparsely returning builder can only re-root and cannot sustain progress, see the "'One hop' means re-rooting" item below). **There is no censorship floor**: the former backstop by any-key forced-only blocks no longer holds, and recovery depends entirely on scheduled builders returning. An outage of all participants (for example a network-wide client bug) is a matter for social recovery and is out-of-model, as it is for any chain. A targeted DoS against near-future lookahead builders merely postpones recovery by $\approx 1/\mathrm{p}$ slots, which is of the same order as the normal residual risk of §3.2 and is not a new hole.
- **Deletion list** : the entire §6.4 episode — announcement/$\mathrm{B\_ra}$, existence challenge/$\mathrm{B\_ch}$/deposit settlement, parent pinning/re-pinning, freshness floor/$\mathrm{s\_base}$/$\mathrm{s\_ra}$, $\Delta\mathrm{\_cont}$ continuation authorization, and the episode state machine — is deleted in full. The review longer apply. §11 is amended accordingly.
- **The recovery accounting is given in §8** (one hop + one round of proving, including a single hop for a long stall; L1-sync catches up according to the backlog). The residual risk is given in §5.4 (single-block orphaning via the landing head $\leq$ the unlanded tail $\leq \Delta\mathrm{\_lag}$, which takes effect only under a colluding lander and has the same bound as the existing worst case). The lander's tail-selection strategy is given in §5.2.

6.5 Aggregator Seat and Economics

- The seat is produced by a perpetual auction (inheriting the v15 §4 skeleton: bonded bidding, the standby queue, the q delayed transition, bonds, and re-auction upon $\mathrm{T\_max}$ expiry — all carried over, with the semantics of q and $\mathrm{T\_max}$ unchanged).
- **The fee source must not form a circular dependency with liveness**: if the service fee comes from the protocol fee stream, then when the chain is idle the fee stream dries up and creates the loop "underpayment → rationally going offline → dependence on the fallback"; the fallback reward is therefore paid solely from the aggregator's bond (already the case, §6.3), and the design of the service-fee source (§11 item 4) must guarantee that the aggregator still breaks even when the fee stream is insufficient (for example, a minimum

service fee backstopped by the treasury). The combination of winning the seat cheaply and then slacking off is already closed off by the strike mechanism of §6.3 (slacking off = strikes = replacement), so all that needs to be handled here is the "honest but underpaid" case.

- **The direction of payment is reversed, stated explicitly**: in v15 the seat paid the treasury because it held an exclusive sequencing right; in this design the seat is a service post (batching + proving + L1 gas), and what is auctioned is the lowest service fee rate (a reverse auction, with the service fee paid out of protocol fee revenue), with the bond accounted for separately. The bidding dimensions are the pair (service fee rate, bond): the lower rate wins, and at equal rates the higher bond wins. Refinement is a calibration item of §11.
- MEV and priority fees go to the builder of each slot (coinbase). The aggregator does not touch sequencing and therefore structurally has no MEV position — this is intentional.
- **Base-fee share — a positive incentive to land a longer chain** : a fixed proportion φ_land (a §11 calibration item) of the sum of the base fees of all blocks in the candidate batch serves as the landing reward and is credited, in the atomic commit at the §5.6 window close, to the beneficiary address of the closing winner (the beneficiary is part of the proof's public inputs, per the anti-plagiarism clause of §5.6, so mempool plagiarism cannot change where it points); the remaining base fee follows the current protocol's destination. Three immediate corollaries: (i) the base of the share grows monotonically with chain length — landing a longer chain yields a larger share, which closes the loop with §5.2's "landing the longest visible chain is the rational choice"; (ii) a provisional lander superseded by a heavier candidate receives nothing (winner takes all) — the opportunity cost of landing a short chain = the entire share + the gas and proving fees already paid; (iii) the share is accounted atomically at close together with the winner's final tuple, and requires no extra transaction or after-the-fact claim. The payment point is the close, not the provisional landing (one point of deviation from the design decision of "sharing directly at landing time"; see Appendix A-4): a provisional candidate can be superseded within the window, so paying on landing would require a clawback from the superseded lander, whereas paying at close is equivalent to "the last step of the landing procedure settles the accounts automatically" and needs no clawback — the cash flow of users and landers differs only by one $\mathrm{W\_settle}$. The existing §6.3 rule that "the reward is charged by the number of slots that a landing advances" (reward anti-grinding) continues to apply to the service-fee component; the base-fee share is a content-volume incentive layered on top of it, and the two point in the same anti-grinding direction (a micro-batch neither suppresses the fallback window nor has much of a share base).

7 Anchoring and the bridge (L1 $\rightarrow$ L2)

- The current per-block anchoring pattern is carried over: the first transaction of every block is the anchor transaction (anchor tx), which references an L1 block at depth $\geq \mathrm{D\_anchor}$ (32 L1 slot), is non-decreasing relative to the previous anchor, and whose freshness does not exceed the cap (the numerical values carry over the shape of the v15 §6.6 constraints).
- **The anchor may not be later than this block's slot time — the L1$\rightarrow$L2 causal-ordering invariant** : in addition to being non-decreasing and fresh, it must also hold that $\mathrm{the\ anchor's\ L1\ timestamp} \leq \mathrm{the\ L2\ timestamp\ corresponding\ to\ this\ block's\ slot}$ (equivalently, $\mathrm{L2\_slot(anchor.L1\_timestamp)} \leq \mathrm{slot}$), and this applies to all blocks alike. Why it is necessary: §6.1

rule 3 only blocks "future slots", and §7 only blocks "an anchor that is too old", and neither blocks "an old slot paired with a new anchor" — a builder could hold back an old slot, then pick after the fact a fresh anchor that has just arrived and that already contains some new L1→L2 message, and sign a tier (i) continuation that consumes that message into an L2 block whose slot timestamp is earlier than the L1 block the message originated in; a Byzantine aggregator lands it ahead of honest recovery → L1→L2 causal ordering is broken, and the bridge/message deadline semantics based on $\mathrm{block.timestamp}$ are distorted. With this invariant added, a block can never reference L1 state from after its own slot time. It does not affect recovery: a recovery block references an L1 block at depth $\geq \mathrm{D\_anchor}$ ($\approx$ 6.4 minutes ago), whose timestamp is always $\leq$ the current wall-clock slot time, so an honest block satisfies it naturally; this invariant rejects only the malformed combination "old slot + new anchor". The L1 entry point and the circuit both prove it ($\mathrm{anchor.L1\_timestamp}$ is a pure function of the referenced L1 block header).

- The time geometry of the anchor's age along the worst-case path (the origin of the $\mathrm{D\_anchor\_max}$ setter invariant, Appendix C P9a):

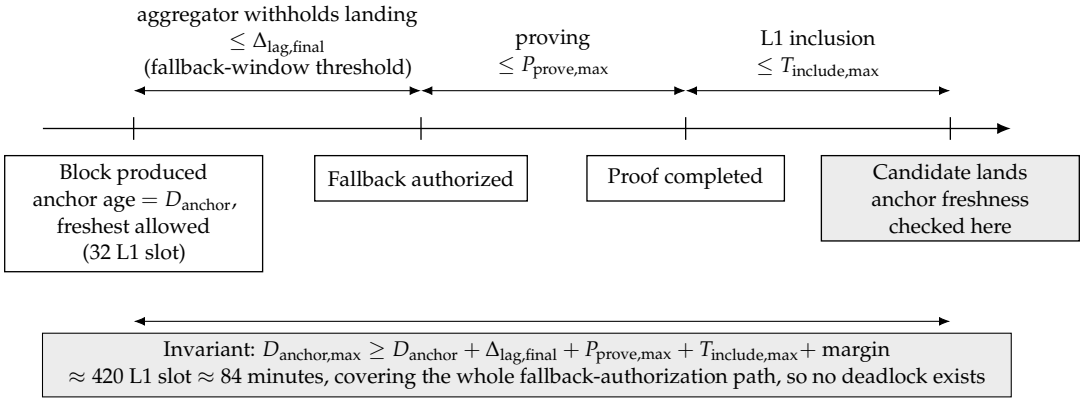

Figure 9: The time geometry of anchor age along the worst-case path: the freshness window $D_{\text{anchor,max}}$ must cover the whole route from block production, through the aggregator's withholding until the threshold authorizes fallback, and on to proving and L1 inclusion. This relation between parameters is exactly the solution to the stale-anchor tail deadlock.

- **The solution to the stale-anchor tail deadlock —** $\mathrm{D\_anchor\_max}$ **covers the fallback authorization period (the commitment layer has been deleted)**: when a malicious aggregator withholds landing for a long time, the anchor of an early block of an honest tail will, "at landing time", exceed $\mathrm{D\_anchor\_max}$ → the tail can then neither be landed (its first block is past the deadline) nor be skipped ($\mathrm{parent\_hash}$ runs through it). The solution is parameter geometry, not a new mechanism: the setter invariant $\mathrm{D\_anchor\_max} \geq \mathrm{D\_anchor} + \Delta\mathrm{\_lag,final} + \mathrm{P\_prove,max} + \mathrm{T\_include,max} + \mathrm{margin}$ (the lag term is $\Delta\mathrm{\_lag,final}$ — the fallback opens its window on $\mathrm{lag\_final}$, so the authorization point is later than under the old $\Delta\mathrm{\_lag}$, and the freshness window must cover it, or else the C1 fix would in turn break this geometric solution) — that is, the freshness window must cover the entire course "from block production, through $\mathrm{lag\_final}$ crossing the threshold and opening the fallback, to the fallback proving and landing it". Consequently, any tail that the fallback is authorized to land (produced while $\mathrm{lag\_final} \leq \Delta\mathrm{\_lag,final}$ and landed by the fallback after the threshold is crossed) necessarily still has a fresh anchor, and no deadlock exists. A rough calculation with the initial parameters: 32 + 288 + 75 + 10 + margin $\approx$ 420 L1

slot (≈ 84 minutes), which is still a deployable value. For situations beyond that window (the fallback too is persistently absent for > D_anchor_max), the stale prefix can no longer be landed and the chain is rebuilt from the window-final head — the same conditional residual risk as §6.3 fallback feasibility, stated honestly. The causal-ordering invariant (the previous item) is unaffected.

- Because there is a block every second (under normal conditions), the intake cadence of L1→L2 messages is on the order of seconds — better than the fallback path of a forced-inclusion-specific cadence. During a gap, messages are consumed by the next honest block; inbound messages have no alternative entry route under a full cartel and can only wait for scheduled builders to resume producing blocks (§8).
- **The L1→L2 message-processing cap** $\mathrm{C\_anchor}$ **— bound to the [processing cursor], not to the anchor reference** : distinguish two independent quantities:
  - **The anchor reference** (which L1 block the block references as its freshness baseline): still non-decreasing + required to be fresh ($\mathrm{landing\ L1\ block\ height} - \mathrm{anchor} \leq \mathrm{D\_anchor\_max}$). A recovery block references a current, fresh anchor — even if the anchor of its parent (a landed head from days ago) is very old, this one large monotone jump is permitted (it merely references a recent L1 block and satisfies freshness), and the anchor itself carries no per-block advance cap.
  - **The L1→L2 message-processing cursor** $\mathrm{m\_consumed}$ (hence $\mathrm{m\_consumed}$ = the global sequence number in the inbound message queue, or the equivalent $(\mathrm{L1\ height}, \mathrm{tx\ index}, \mathrm{log\ index})$ triple): the unique canonical algorithm = [the uniform maximum prefix]: L1→L2 messages form a public FIFO queue (the contract maintains a queue root + tail cursor, kept separate from $\mathrm{m\_consumed}$); every block must consume the longest prefix of that FIFO that satisfies both "entry count $\leq$ $\mathrm{C\_anchor}$" and "cumulative declared gas $\leq$ the L1→L2 message gas share" (restricted to those that have arrived within the height referenced by this block's anchor and are unconsumed; taking that longest prefix is legal, and only taking less than it is illegal — as in §5.1 rule 4, with no second "exactly some entry count" criterion). A recovery block references a fresh anchor (spanning days of L1), each block digests this prefix, and the remainder is continued by subsequent blocks ($\mathrm{m\_consumed}$ advances monotonically). The backlog catch-up bound is correspondingly two-constraint: $\mathrm{backlog\ drain\ time} = \max(\mathrm{ceil(entry\ count\ /\ C\_anchor)}, \mathrm{ceil(cumulative\ gas\ /\ the\ G\_l1msg\ share)})$ blocks (the bounds of §6.4/§8 are to be read this way). Without this lower bound, $\mathrm{m\_consumed}$ is monotone yet may stagnate: a builder that keeps processing 0 messages while still using a fresh anchor could censor L1→L2 messages indefinitely, invalidating the $\mathrm{backlog\ /\ C\_anchor}$ catch-up bound of §6.4/§8. With the lower bound added, the catch-up bound is circuit-enforced and redeemable. Consumption is not execution — an inbound message that is illegal against the preceding state is consumed-and-discarded under the same consume-and-discard rule as §7. **The real anti-censorship bound is** $\mathrm{D\_anchor\_max}$**, not** $\mathrm{C\_anchor}$ **alone**: the maximum prefix only forces "consumption of those that have arrived within the height referenced by this block's anchor", so a builder that pins its anchor at the oldest height freshness permits ($\mathrm{landed\ head} - \mathrm{D\_anchor\_max}$) can legally consume no newer message at all. But this is not indefinite censorship: the anchor must satisfy the freshness lower bound $\mathrm{anchor} \geq \mathrm{landing\ L1\ block\ height} - \mathrm{D\_anchor\_max}$ and be non-decreasing, so as the landed head advances, that lower bound rises, and a message that arrived at height h is forced into "within the anchor-referenced height" once

$\text{landed head} - \text{D\_anchor\_max} > \text{h} \rightarrow$ it must then be consumed. Hence the worst-case inclusion delay of an L1→L2 message ≈ D_anchor_max (freshness forces the anchor frontier forward) + $\text{backlog} \,/\, \text{C\_anchor}$ (the processing rate once it has entered the to-be-consumed set); the two parameters jointly bound it, not C_anchor alone. Why it is split this way: the §4.2 tier (ii) recovery block has to reconnect to the wall clock in one jump from a landed head days old. If C_anchor were imposed on the anchor reference, then the anchor of a recovery block could advance only C_anchor from its stale parent anchor, would still be stale, and would fail the freshness check — failing precisely on the long stall it is meant to support. Once it is rebound to the message-processing cursor: the anchor reference is fresh (satisfying §7) and L2 ordering recovers in one jump; while a huge volume of L1→L2 messages is processed block by block at C_anchor per block, so a full sync takes $\text{message backlog} \,/\, \text{C\_anchor}$ blocks (after recovery, 1 block/second, which is fast). This is the same technique and the same shape as a fresh reference plus a rate-limited cursor. C_anchor is a §11 calibration item.

- **A shared block-level gas budget + a per-message gas cap — sealing off the combined-gas chain stall**: the original risk was that the "forced prefix" and "L1→L2 message consumption", two mandatory obligations each with its own quantity, could together burst the block gas cap and leave no legal block even with everyone honest. With the forced prefix gone that combination disappears, but **one side alone can still reproduce a permanent stall, in a milder form**: if the head of m_consumed is an inbound message whose declared gas exceeds "the block cap minus the fixed anchor overhead", the maximum prefix that fits is empty at every subsequent block. Blocks stay legal — rule 4 asks for the longest prefix that fits, not for a fixed count — but m_consumed can never advance past that message, so the inbound message queue is blocked for good while the chain itself continues. The damage is confined to the bridge rather than to liveness, which is why the per-message admission cap below is the clause that carries the weight (model property P7b constructs exactly this state and confirms the cursor stalls). The first two clauses below are therefore retained, and clauses 3/4 restated for two parties (fully deterministic, verifiable by both the circuit and L1):

  1. **A per-item admission gas cap**: inbound L1→L2 messages get a per-item gas cap at enqueue time, and an over-cap item is refused admission — this is the load-bearing clause against one-sided deadlock, and it is not relaxed just because the forced queue is gone;
  2. **A shared block-level budget invariant**: $\text{the fixed anchor overhead G\_anchor} + \text{the L1→L2 message gas share} \leq \text{block\_gas\_limit}$, as a setter invariant on the consensus constants; the message side's "guaranteed capacity after deducting the anchor overhead" always fits into the block;
  3. **Deterministic priority and overflow**: the enforced execution order within a block = anchor → L1→L2 message prefix; following m_consumed, the block "consumes up to its guaranteed capacity, and the remainder deterministically overflows into subsequent blocks" — the cursor is advanced only by legal blocks, so a backlog buys delay and cannot buy a chain stall;
  4. **A watermark for lowering parameters**: when C_anchor or the message gas share is lowered, the new share must be $\geq$ the maximum gas among the unconsumed entries of the message queue, guaranteeing that in any reachable state there is no item that is "already enqueued yet unable to fit within the guaranteed capacity". The two sepa-

rately rate-limited obligations are thereby coordinated by one shared budget, and can no longer be combined into a "no legal block" deadlock. The numerical gas shares are folded into §11.

8 What happens when each role goes offline (liveness accounting, in a single table)

Who goes offline	**What the user sees**	**Recovery mechanism**	**Recovery time**
A single builder	Its slot becomes a gap; every other slot proceeds as usual	No recovery is needed; the next slot continues naturally	~1 second
Consecutive absences $\leq \mathrm{G\_max} - 1$ (63 slots, so the next block's parent distance is $\leq \mathrm{G\_max}$)	No new blocks for a few seconds	The next builder resumes under §4.2 tier (i); an absentee that supplies its signature after the fact can also bridge the interval	~number of offline builders $\times$ 1 second

Who goes offline	**What the user sees**	**Recovery mechanism**	**Recovery time**
Consecutive absences $\geq$ G_max slots, extending to hours or days (catastrophic stall)	No new blocks for the duration of the stall	**Land the best chain first, then recover from its final tip (§5.2/§6.4)**: if a frozen long tail existed before the stall, the lander lands it first (thereby honoring its preconfirmations), and once it has reached F_l1 the recovery block takes its tip as parent and is produced under §4.2 tier (ii); where there is demonstrably no better chain, the recovery block is itself the best chain and lands directly. Landing a worse candidate is superseded inside the §5.6 window by a heavier one (self-defeating)	**Ordering ≈ one round of proving + landing (≈10–15 minutes) + the** W_settle **window close + in the worst case an additional wait of** $\leq$ F_l1 **+** D_anchor **when the stall begins immediately after a landing** ; where a frozen long tail exists, this includes one round of proving to "land the tail first" and the time to publish that tail's bodies, which is linear in its size but proceeds in parallel across L1 blocks at L1's blob bandwidth rather than one transaction per settlement window (§6.1) — for a tail of any plausible size this is minutes, not windows; the L1-sync backlog catches up at C_anchor per block; **sustained progress requires scheduled builders to keep returning at a window density of** $\geq$**1/**G_max **(the normal-mode** G_max **liveness floor)** — With no forced inclusion, there is no forced-only floor to serve as an alternative

Who goes offline	**What the user sees**	**Recovery mechanism**	**Recovery time**
Aggregator goes offline (honest but offline)	**Nothing perceptible** (preconfirmations proceed as usual); finality is deferred	$\mathrm{lag} > \Delta\_\mathrm{lag} \rightarrow$ the fallback window opens, anyone may land and a strike is recorded; $\mathrm{m\_agg}$ strikes $\rightarrow$ the standby is promoted	Finality lag is pinned at Δ_lag + the fallback response; 0 interruption of the sequencing service
Aggregator online but maliciously stalling (Byzantine, landing micro-batches)	As above	Micro-batches cannot hold lag down $\rightarrow$ the fallback window opens, strikes are recorded and the seat changes hands exactly as before (§6.3)	As above (**bounded — but conditional on permissionless fallback being economically viable**); the seat changes hands within $\mathrm{m\_agg}$ strikes
Aggregator offline + no standby	As above	Fallback landing remains available throughout (no seat is required); the normal cadence resumes once the auction has cleared	0 interruption of the sequencing service

Who goes offline	**What the user sees**	**Recovery mechanism**	**Recovery time**
All builders at once (catastrophe or cartel) — **out-of-model** (§1 trust model)	No new preconfirmations; **transactions already in the pool have no entry path at all**	**With no forced inclusion, there is no in-protocol remedy**: the "any-key forced-only block" escape valve is gone and the right to produce a block belongs to scheduled builders alone. The only way out is new builders entering the registry, but they reach the lookahead only after the D_snap (5 epoch) snapshot delay plus window alignment, during which the chain is fully stopped	**Unbounded** (it depends on when the cartel breaks up, or when new builders are scheduled in); the latter alone is $\geq D\_snap$ + window alignment
Builders and aggregator colluding as a whole (block bodies kept private) — **out-of-model** (§1 trust model)	Preconfirmations are still issued but cannot be trusted; finality stalls, and **correctly** no aggregator strike is produced (the stall is attributed to the cartel)	**No defense-in-depth exit** (This design does not include forced inclusion); the only self-healing route is a single builder defecting from the collusion and producing a landable fork (§5.5)	**Unbounded**; everything waits for the collusion to break down

Who goes offline	What the user sees	Recovery mechanism	Recovery time
Proving-system failure	Preconfirmations proceed as usual; no new finality	**Zero false positives for fallback strikes holds** (nobody can land ⇒ no strike is recorded, §6.3); but the exemption for late fees and late counts **does not yet exist** — until §11 item 8 (blocking) is complete, the first batch landed after recovery is charged under the current rules	Equal to the duration of the failure; the risk of wrongly penalizing the recovery batch remains open until §11 item 8 is closed

By comparison with v15: there, "one seat goes offline" = a 20–26 minute service interruption (when a standby exists); here the same class of event costs 1 second (a builder) or 0 seconds (an aggregator, which affects only the cadence of finality). This is the reason this design exists.

9 Master table of slashing and bonds

Fault	How it is adjudicated	Consequence
Builder equivocation (two block headers for the same slot)	A challenger submits the two signed block headers and the L1 contract verifies the signatures directly	Slash L_eq ($\geq$80% burned, the remainder paid to the submitter) + removal from the registry

Fault	How it is adjudicated	Consequence
Builder skips the previous block / is absent	Not a fault (producing a block is an opportunity, not an obligation)	No slashing; a skip is **profitable** for the skipper (fee/MEV transfer). Orphaning depth follows the three tiers of §5.4: an adjacent skip affects a single slot; under **tier (i)** a single-block deep skip orphans at most $G\_max - 1$ unlanded blocks, and a minority coalition relaying a sparse branch can reach the entire unlanded tail $\approx \Delta\_lag$; under **tier (ii)** a single malicious builder can, with one block built on a final landed head, orphan at most the entire unlanded tail $\approx \Delta\_lag$. The worst-case depth is the same in all three tiers, namely $\leq$ **the actual length of the unlanded tail (**$\approx \Delta\_lag$ **when the fallback lands on schedule, conditional)**; in every tier the attacker must beat any heavier honest candidate inside the §5.6 window before it can become final, the signature is on record for public attribution, and the frequency is constrained by the honest-majority assumption; an absentee forfeits the fee income of its own slot
Builder signs a header but withholds the body (withhold-body, §5.5)	Not adjudicable ("it has the body and is not releasing it" cannot be proven)	No slashing; the block is certain to be skipped and the withholder gains nothing at all; the receipt is itself public evidence, and the response is reputational
Aggregator dereliction (a successful landing by someone else while the fallback window is open = a strike, §6.3)	Mechanically adjudicated on L1 (the lag state plus the fact that a fallback batch was accepted)	Each strike is penalized at the liveness tier; m_agg accumulated strikes $\rightarrow$ termination + promotion of the standby
Aggregator takes a prefix / lands a worse chain (truncating the suffix)	**Not a slashable fault**	No slashing; landing a worse chain is **self-defeating**: inside the §5.6 settlement window anyone can land a heavier proven candidate and supersede it, so the malicious party has wasted its gas and proving fees; and if only a prefix is taken, the suffix can still enter a subsequent window
Anyone submits an invalid batch	The proof fails verification and the transaction reverts	Wasted gas, no protocol consequence
Builder signs a block that cannot execute (a body that does not apply, or a state root that does not match)	**Not adjudicable on L1** without a proof of invalidity, which this design does not define; the block simply fails to be provable	No slashing. The block can never land, so it cannot corrupt state, and the signer forfeits its own slot revenue and gains nothing — the same shape as body withholding. The residual cost is borne by others: honest successors that have not yet executed the body may build on it within the detection latency, and those blocks are stranded when the next builder re-roots past it under §4.2 tier (i). Unlike equivocation this costs the actor nothing and is repeatable in every slot it is scheduled for, bounded only by its lookahead share w_max. **A preconfirmation issued for such a block is void, and its holder cannot tell from the signature alone**

The structural property of this table: every slashable fault is adjudicated either by direct signature verification on L1 (equivocation) or by a mechanical judgment over the L1 clock and L1 facts (landing strikes) — there is no class of slashing that requires an L2 evidence chain plus an arbitration period (the watchtower-dependent *slashing* of the v15 §8b kind does not exist here, because "preconfirmation" and "block" are one and the same signed object, so equivocation is simply signing two headers for one slot and is directly verifiable). One limit of that identity must be stated plainly: the builder's signature attests authorship of a header and the

choice of parent, **not** that the block executes. Executability is established only by the validity proof at landing, so a signed preconfirmation is a commitment to include and to order, not a proof that the block is applicable. There do exist three classes of misbehavior that are not slashable and are only deterred (skipping, body withholding, and signing a block that cannot execute — §5.4/§5.5 and the row above) — this design does not eliminate them; the precise statement: withholding a body yields the actor no benefit, whereas skipping is profitable (the fee/MEV transfer of a single slot), and its frequency is constrained by the honest-majority assumption of §1 — both leave public signed evidence for attribution.

10 Detailed comparison with v15 (for readers who have read v15)

- **What became of v15's nine invariants**: I1 (derivation is a pure function) is inherited and strengthened (timestamps and the lookahead are all pure functions); the spirit of I2 (a single adjudication / computed-state certificate) is inherited by the landing-timeout adjudication, but there is no longer an epoch adjudication object; I3 (the epoch always advances) is replaced by "the L1 chain head can always be advanced by anyone (fallback landing)" — with no forced inclusion; under a full cartel there is no route anyone can advance; I6 (forced content always flows) is **no longer inherited** — This design does not include forced inclusion, so this invariant has no counterpart here; I7 (sealing is sender-agnostic) is generalized into "landing carries no authority"; I8 (pre-committed beneficiary) is inherited (the recipients of the fallback reward and of the slashing bounty are made public inputs of the proof); I9 (safety slashing is denominated in ETH) is inherited by $\mathrm{L\_eq}$.
- **v15's review legacy is not voided**: the auction skeleton (§4), the at-fault-pays model (§6.7), the anchor freshness constraint (§6.6), the burn-dominant split of slashed funds (§8) and the discipline of invariant checks in parameter setters are all carried over directly. The parts of v15 that revolved around "one irreversible adjudication per epoch" (EBC, AC, default derivation, anarchy mode, cancellation cascade) have no counterpart in this design, so the corresponding historical review findings do not apply.
- **Response to the four criticisms in the assessment document**: the single point of failure is merely relocated → resolved (landing carries no authority, plus lag-based fallback and strikes, which are bounded for both the offline and the Byzantine aggregator — §6.3); builder faults are all of the watchtower-dependent class → the slashable class is resolved (equivocation is verified directly on L1), but two classes of non-slashable misbehavior (skipping, body withholding) remain as deterrence-grade residual risk and are stated explicitly (§5.4/§5.5) — this is "downgraded and explicitly priced", not "eliminated"; fair exchange has become load-bearing → partially resolved (intermediate deletion is eradicated by the signature chain; the residual risk is as above); the epoch mechanism needs a rewrite → conceded, and this document is that rewrite, whose result is shorter than v15.

11 Open items

1. **Builder-set size and admission rules**: $\mathrm{N\_max}$, the weight cap $\mathrm{w\_max}$ on the bond-weighted sampling, the rules for excess competition, and liveness requirements (whether prolonged absence should reduce weight).
2. **Costing the separated data path**: §6.1 specifies bodies as blob transactions referenced by versioned hash. What remains is quantitative — the gas cost of registering versioned

hashes and of checking references at the entry point; whether blob-commitment opening is cheaper inside the circuit or on L1 through the point-evaluation precompile, at the segment sizes this protocol needs and with openings batched across many blobs; and who funds $\mathrm{postData}$ in steady state, the natural candidate being the aggregator out of the §7 base-fee share. Reimbursement for a fallback party's data posting is specified — φ_data per published byte the winning chain consumed (§6.3) — but φ_data must be indexed to the blob market rather than fixed, or a blob-price spike makes permissionless fallback unprofitable exactly when an aggregator withholding data is the reason fallback is needed.

3. **Residual questions on window-scoped slashing**: §4.3 now scopes effectiveness to settlement windows, which removes the equivocator's control over the timing but leaves two items to settle. The slashed key may still contribute blocks for up to one $\mathrm{W\_settle}$ after the slashing becomes effective; whether that window should be shortened by having a slashing also close the open settlement window early is a design choice with its own interaction with §5.6. And the argument that readmitted backfilling is harmless — that a backfilled block still occupies the signer's own scheduled slot, still chains through $\mathrm{parent\_hash}$, and still has to win the §5.2 order — deserves one dedicated adversarial pass before it is relied upon.
4. **Calibrating** $\mathrm{C\_fixed}$**,** φ_adv**,** φ_data **and** $\mathrm{R\_window\_max}$: the §6.3 rule meters fixed cost per $(\mathrm{count},\ \mathrm{tip\_slot})$ level and relies for its anti-splitting property on $\mathrm{C\_fixed}$ being set at or just below the true indexed cost of one proof plus one inclusion. That constraint has to be met by an indexing procedure that tracks proving and gas markets without being manipulable by the parties it pays, which is the substantive open question here; setting $\mathrm{C\_fixed}$ above true cost makes prefix-splitting profitable, and setting it far below makes honest fallback unprofitable. $\mathrm{R\_window\_max}$ must be placed far enough above a plausible honest window that it never binds in normal operation, and its binding should raise an alarm rather than pass silently.
5. **Pricing of** $\mathrm{L\_eq}$: a method for estimating the worst-case extractable value of a single slot, and whether governance should adjust it as market conditions change.
6. **Landing parameters**: $\Delta\_\mathrm{lag,prov}/\Delta\_\mathrm{lag,final}$, $\mathrm{m\_agg}$, δ_slash, and the exponential cost cap on the fallback reward; **recovery parameters** : $\mathrm{F\_l1}$ (the L1 finality depth of the final landed block in tier (ii)), $\mathrm{C\_anchor}$ (the per-block cap on the number of entries consumed by the §7 message-processing cursor $\mathrm{m\_consumed}$; not a cap on anchor advancement, since anchor references have no per-block cap), Δ_prop (the propagation settling amount of the §5.2 landing depth, normally « Δ_lag) and $\mathrm{D\_anchor\_max}$ (the anchor freshness cap). $\mathrm{D\_anchor\_max}$ **setter invariant:** $\mathrm{D\_anchor\_max} \geq \mathrm{D\_anchor} + \Delta\_\mathrm{lag,final}(\text{converted to L1 slots}) + \mathrm{P\_prove,max} + \mathrm{T\_include,max} + \text{queue/clock margin}$

(the contract setter implements only this single strongest invariant. $\mathrm{D\_anchor}$=32 L1 slot; $\mathrm{P\_prove,max}$=the worst-case proving latency; $\mathrm{T\_include,max}$=the bounded-inclusion bound of §1. The other setter relation: $\mathrm{T\_include,max} < \min(\mathrm{W\_settle} - \mathrm{P\_prove,max} - \mathrm{margin},\ \mathrm{D\_anchor\_max} - \mathrm{D\_anchor} - \Delta\_\mathrm{lag,final} - \mathrm{P\_prove,max})$) — Otherwise the recovery block (and any block) would have its anchor expire inside the proving-plus-landing window and would have to be re-signed and re-proven (§6.4, anchor freshness clause); this is the precondition for recovery "not expiring because of a gap", and it replaces the 64-second structural window of the old $\mathrm{s\_ra}$ with a free parameter that can be set large enough. Stated honestly: a finite $\mathrm{D\_anchor\_max}$ is still a "proving + inclusion delay vs. freshness window" trade-

off, only with a window that can be configured large enough, which is why C1 is explicitly conditional on the bounded-inclusion assumption "temporary L1 congestion or censorship $\leq$ D_anchor_max" and claims nothing unconditional against censorship of arbitrary length. **Settlement-window parameters** : W_settle (the length of the settlement window, counted in L1 blocks; setter invariants: the lower bound W_settle $\geq$ P_prove,max + T_include,max + margin, and the consensus upper bound W_settle_max (which includes a margin for wall-clock conversion under missed L1 slots), with an initial suggestion of $\approx$ 100 L1 slot $\approx$ 20 minutes); the L1-reorg rollback of window state is folded into item 9. The semantics of final_ref (a §4.2 tier-(ii) block-header field, attesting that the parent was already F_l1-final at signing time, already part of the block-header tuple and of §5.1). Δ_prop is demoted to an off-chain strategy reference for the completeness of a lander's view (§5.2, not an on-chain attribution parameter). (C_force/C_bridge/F_delay/H_force left with the removal of forced inclusion.) Shared gas shares (§7): G_anchor and the L1→L2 message gas share, subject to sum of the two $\leq$ block_gas_limit + a per-entry admission cap + a watermark. (The Δ_ra/Δ_ra_ext/B_ra/B_ch/Δ_cont of the old re-anchor episode were removed together with the episode and are no longer parameters.) The behavior of the strike mechanism under extreme congestion (an L1 fee spike that leaves nobody willing to perform the fallback). **The concrete bounds given around these parameters in §7 and §8 (**backlog/C_anchor**, recovery time, the** D_anchor_max **anti-censorship bound and so on) are all [provisional values] until the parameters have been calibrated** — the form is settled, the numbers are not.

1. **Details of the aggregator reverse auction**: the funding source for the service fee rate (protocol fees or the treasury), the bid-evaluation function over rate and bond, and the bindingness of the standby queue.
2. **P2P layer specification**: the propagation protocol for signed blocks, the engineering guarantees for block-body availability (the format of preconfirmation receipts, the relaying obligations among builders, whether erasure-coded propagation is worth it) and public metrics for skipping and body withholding (published as a public dashboard, the lever that makes deterrence bite — the deterrence-grade residual risk of §5.4/§5.5 rests entirely on it).
3. **Proving-circuit cost**: an assessment of the in-circuit cost of per-block signature verification (at most 384 signatures per epoch) plus recomputation of the lookahead; whether BLS aggregate signatures are worth introducing (replacing per-block ECDSA with an aggregatable signature, lowering both the circuit cost and the potential cost of direct verification on L1 — this bears on the choice of builder key scheme); and incremental continuation and aggregation for batch proofs. **This is a pre-implementation liveness gate, not ordinary follow-up work**: once the chain head has been advanced by a small batch, a batch already being proven against the old head must be able to reuse the proven prefix and continue from it rather than be re-proven from scratch. Two liveness arguments in this document rest on that capability — that fallback micro-batches cannot invalidate an aggregator's in-flight catch-up proof (§6.3), and that marginal reward metering stays economically viable across successive improvements — and both remain conditional until it is demonstrated. Extending one's own proof is not the same operation as reusing someone else's, so the gate must separately establish, with latency bounds for each: (i) appending blocks to an identical already-proven prefix; (ii) changing the candidate's beneficiary or other public inputs; (iii) extending a competing fork from the same frozen baseline; (iv) reusing a proof whose original submitter may never publish its auxiliary prover state; and (v) aggregation

as distinct from recursive continuation. Until those are shown, the claim that drip-feed stalling has lost its teeth is conditional.

4. **Concurrent multi-batch landing and reorgs**: how to handle the gas competition when several parties race to land inside the fallback window (expected to follow the v15 §11.3 conclusion that "the competitors bear their own costs and the chain advances as usual", to be re-checked).
5. **Exemption mechanism for proving-system outages (blocking)**: the form follows v15 §10.4 level 3 (independent corroboration + a bounded exemption + an H_toll_max-style cap), substantially simplified in the absence of the cancellation cascade, but it requires a dedicated derivation of its own; until that is done, the "zero false positives" claim of §6.3 is limited to fallback strikes.
6. **Handling of L1 reorgs**: the rollback semantics for a landed batch that meets an L1 reorg (expected to be considerably simpler than v15 §3.1/§13-S.1, because there is no cross-transaction certificate state machine, but it still has to be written down).
7. **Migration path**: the upgrade ordering from the current deployment, and from v15, to this design.
8. **Interaction sequence diagram**: the interaction of batch landing, equal-height fork resolution and the fallback window is currently spread across §5–§7, and deserves a single normative sequence diagram.
9. **Availability and cost of fallback provers**: to be folded into the threat model — the Byzantine-aggregator liveness bound of §8 is conditional on it; and the design of fallback incentives under extreme congestion or when the proving cost exceeds the reward.
10. **Worst-case calibration of the margin on the lag thresholds**: $\Delta\_lag,prov$ (4 epoch ≈25.6 min) leaves only ≈5.6 min of margin over the upper bound of the normal provisional lag band (20 min); for $\Delta\_lag,final$ (≈9 epoch ≈57.6 min), the margin over the steady-state lag_final band (≈35–40 min) is carried by the margin term of W_settle_max. A proving spike compounded by jitter in L1 finality could push a conscientious aggregator past the threshold and open the fallback window in error. Whether to widen it needs to be assessed.
11. **Genesis and bootstrap semantics**: the initial lag (with no landed head, lag may be arbitrarily large and the fallback window is "open" from genesis onwards), the selection of the first aggregator, and strike accounting before the first batch has landed are all undefined. A genesis anchor (a genesis landed head) plus rules suppressing fallback and strikes during the bootstrap period must be defined.
12. **Strengthening of the formal specification — the pre-implementation gate** : an executable reference model of the §5.2 total order and the §5.6 window state machine, plus property tests — see Appendix C and $settlement\text{-}window\text{-}model.py$; any change to the rules must be mirrored in the model and the model re-run, and a specification whose model no longer passes has a defect in it. The model covers P1–P13 (the total order, the window state machine, cursor and gas arithmetic, bridge reservation, anchor geometry and causal ordering, window-scoped slashing effectiveness, the fallback accounting snapshot and the fallback reward metering; 34 assertions, all passing). Two items below remain open regardless: (b), the precise definition of the lookahead sampling operator, must be closed before election is implemented, and final acceptance requires a human safety review — this gate passes nothing automatically. The original list: (a) a single set of pseudocode or state machines for block legality / parent selection / the best-chain total order (the §5.2 triple,

including irreflexivity, totality and transitivity property tests) / the settlement-window $\mathrm{openWindow}/\mathrm{acceptCandidate}/\mathrm{replaceCandidate}/\mathrm{closeWindow}$ state machine (§5.6, including double-candidate supersession, baseline freezing when the forced or message queue is non-empty, and L1-reorg and lazy-close tests.6) / the shared gas budget and overflow (§7) / atomic rollback of $\mathrm{landed\ head}$ + state root + $\mathrm{m\_consumed}$ + lag under an L1 reorg (§11 item 9) (the current prose relies heavily on review notes and cross-references, and the P1-level parent-block / fork-choice / lander interactions are easily missed by a reader); (b) the precise definition of the deterministic weighted sampling algorithm for the $\mathrm{lookahead}$ —.2 gives a complete executable candidate operator (Python code using a capped weight prefix sum plus positioning by seed modulo total weight, verifiable by running $\mathrm{lookahead\text{-}model.py}$), and this item closes once that choice is confirmed that operator; (c) terminologically distinguishing L2 acceptance finality from L1 finality $\mathrm{F\_l1}$ with different words (the conflated use of "final" makes the two-tier parent rule harder to reason about).

Master parameter table

Parameter	Initial value	Unit	Key invariant / source
slot	1	second	§3.1
epoch	384	slot	§3.1
$\mathrm{W\_size}$, lookahead window	384	slot (1 epoch)	§3.2; a fixed aligned partition. No invariant constrains it: the guaranteed horizon is set by $\mathrm{D\_snap}$ and $\mathrm{F\_final}$, not by the window
$\mathrm{H\_look}$, lookahead horizon (published floor)	768	slot (≈2 L1 epoch)	§3.2; the true guarantee is $12 \times (\mathrm{D\_snap} - \mathrm{F\_final})$ = 1152 slots, so this floor carries 384 slots of slack
$\mathrm{D\_snap}$, snapshot delay	5	L1 epoch	$\geq \mathrm{H\_look}/12 + \mathrm{F\_final} + \mathrm{margin}$
$\mathrm{w\_max}$, weight cap	20%	—	§3.2
$\mathrm{N\_max}$, registry capacity	64	addresses	§4.1
$\mathrm{G\_max}$, gap cap (tier (i): gap ≤ G_max, independent of the parent's landing status)	64	slot	§4.2; a depth knob, not a hard cap
$\mathrm{G\_max\_landed}$ (tier (ii): no cap on the gap, but the parent must be an L1-final landed block)	∞ (no cap)	—	§4.2 the two-tier parent rule
$\mathrm{F\_l1}$, final depth	TBD	L1 slot	the finality depth of a tier (ii) parent block, §4.2
$\mathrm{C\_anchor}$, cap on L1→L2 message processing	TBD	messages/block	bound to the $\mathrm{m\_consumed}$ cursor in §7
$\Delta\mathrm{\_prop}$, landing propagation settling	TBD (« $\Delta\mathrm{\_lag,prov}$)	slot	the §5.2 landing depth
$\delta\mathrm{\_slash}$, delay before a slashing takes effect	64	slot	§4.3
$\Delta\mathrm{\_lag,prov}$, service observation threshold	4	epoch (≈25.6 min)	$\geq$ upper bound of the normal provisional lag band, §6.2/; does not enter the penalty rules

Parameter	**Initial value**	**Unit**	**Key invariant / source**
$\Delta\_{lag,final}$, fallback/strike threshold	≈9 (= $\Delta\_{lag,prov}$ 128 + W_{settle_max} 150 + margin 10, counted in L1 slots = 288)	epoch (≈57.6 min)	in steady state $lag\_final \approx lag\_prov + W\_settle$ (35–40 min) $>$ old threshold, so unless it is raised the fallback window is permanently open; §6.2/§6.3
W_settle_max, consensus upper bound on the window	TBD (≈1.5×W_settle)	L1 block (including a wall-clock conversion margin)	pins down the revocable exposure and $\Delta\_{lag,final}$, §6.2
G_strike, strike rate limit	1	epoch	
m_agg / m_agg'	2 / 4	strikes	termination thresholds, §6.3
D_anchor / D_anchor_max	32 / TBD (≈420)	L1 slot	anchoring depth / freshness cap; $D\_anchor\_max \geq D\_anchor + \Delta\_{lag,final}$(L1 slot) $+ P\_{prove,max} + T\_{include,max} + margin$
W_settle, settlement window	TBD (≈100 L1 slot ≈20 min)	L1 block	§5.6: $\geq P\_{prove,max} + T\_{include,max} + margin$ the settlement-window design
φ_land, base-fee share ratio	TBD	—	the sum of the base fees of the candidate chain × φ_land is credited to the winner's beneficiary at the close, §5.6/§6.5
gas shares G_anchor/$C\_l1msggas$	TBD	gas	§7: the sum of the two $\leq block\_gas\_limit$ + a per-entry admission cap + a watermark

(The values are initial suggestions; those marked "TBD" are calibration items under §11. Unit conversion: 1 L1 slot = 12 s = 12 L2 slot.)

Appendix A: Design alternatives considered and rejected

Each item states an option that was weighed against the choice this document makes, and why the choice went the way it did.

1. **The lookahead randomness does not mix in a proof hash.** A natural alternative is "L1 consensus-layer data + the hash of the most recent proof". This document takes only the former. The reason (§3.2): a proof hash is entropy that the party producing the proof can re-grind at zero cost, so mixing it in hands partial control of the lookahead to the aggregator, which runs counter to the trust-minimization goal. The 1-bit-level bias residual of pure L1 beacon randomness is acceptable for the purposes of the lookahead.
2. **Parent linkage uses the block-header hash, not the parent's signature.** Both are workable; this document makes the hash the consensus rule (it is structurally necessary and transitively commits to the entire history) and demotes carrying the parent's signature inside the child block to an optional evidence-packaging convention (§4.2).
3. **The residual risk from skipping is classified as deterrence-grade and explicitly accepted.** The signature chain eliminates intermediate deletion by the aggregator, but a skip by the adjacent builder is not falsifiable inside the protocol (§5.4). This document chooses to say so plainly and to handle it with publicity + reputation + a cap of one slot on the loss, rather than introducing an undecidable "but I did publish it" arbitration. If elimination at the structural level is wanted, the known price is publishing a commitment to L1 every slot (fees revert to what they were before aggregation saved them) — not recommended. (This sentence is precisely the cost identity cited in §1: the fee for buying protocol-level commitments or DA for the pre-landing stage $\approx$ the fee this design saves by landing in aggregated batches — buy the former and the latter's savings cease to exist.)
4. **The payment point for the base-fee share: the close, not provisional landing.** The obvious alternative is to distribute the base fee directly to the aggregator at landing time. This document chooses to account for it in the atomic commit at settlement-window close (§6.5). The reason: a provisional landing can be superseded inside the window by a heavier candidate (§5.6), so paying at landing would require designing a clawback mechanism for the superseded party — that is a new state machine and a new attack surface. Paying at the close, by contrast, is part of the atomic commit at the close, requires zero additional machinery, and happens to implement exactly the "winner takes all" incentive (a superseded lander receives nothing, which maximizes the opportunity cost of landing a short chain). The only price is that the lander's cash flow is deferred by one W_settle ($\approx$20 minutes). Paying at landing would require accepting the complexity of a clawback state machine back into the design.

Appendix B: Glossary

Term	Definition
slot	The 1-second time unit of L2
lookahead	The deterministic mapping from slot to builder address
builder	The preconfer that produces and signs a block according to the lookahead
aggregator	The service seat that packs, proves and lands
signature chain	The unlanded chain linked by parent_hash and signed block by block
landing	A batch plus its proof going onto L1 as a candidate; at window close the heaviest becomes final
batch	A stretch of contiguous signature chain + data + proof
gap	A slot with no legal block
skip	A builder's signed choice not to build on the immediately preceding block that does exist
fallback landing	Anyone landing, and claiming compensation, once the aggregator has timed out
equivocation	Signing two different block headers for the same slot; slashed by direct verification on L1
two-tier parent rule	The criterion is the gap: (i) a gap $\leq \mathrm{G\_max}$ (independent of the parent's landing status); (ii) no cap on the gap, but the parent must be an L1-final landed head
recovery block	A block that, during a stall, is built on an L1-final landed head under tier (ii) and reconnects to wall-clock time (§6.4)
benign stranding	When the parent tip of a recovery block is advanced, that block is voided and simply has to be rebuilt; no state is damaged (§6.4)
consume-and-discard	An inbound message that is illegal against the preceding state still advances the cursor but is not executed
best-chain total order	Lexicographic order on the candidate's own triple $(\mathrm{count}, \mathrm{tip\_slot}, \mathrm{tip\_hash})$; the lane component retired together with forced-only blocks (§5.2)
best-chain landing strategy	The lander lands the best chain under the §5.2 total order — not an obligation: landing a worse chain is superseded inside the §5.6 window by a heavier candidate and is self-defeating
settlement window	Opens once the first candidate extending the window-final head has landed, and closes deterministically $\mathrm{W\_settle}$ L1 blocks later
candidate batch	A landed batch that extends the window-final head and carries a validity proof; inside the window it can be superseded by one that is strictly heavier under the §5.2 total order (§5.6)
window-final	The heaviest proven candidate at settlement-window close; withdrawals and bridging execute only from this state (§5.6/§6.1)

Appendix C: Executable reference model of the settlement window (normative pseudocode + property verification)

The delivery of §11 item 18, "the pre-implementation gate" *. The normative pseudocode is that of this appendix and the body of §5.6; the* ***executable version*** *is in* settlement-window-model.py *(dependency-free Python, runs directly), which prints its verdict when run.* ***Discipline****: any change to the §5.2 total order, the §5.6 window state machine, the §4.3 slashing gate or the §7 cursor and gas rules must be mirrored in the model and the model re-run — the three being out of sync counts as a specification defect (the same practice as the v15 model_checker).*

Properties that have been verified :

Property	Content
P1	The total-order key $(\text{count},\ \text{tip\_slot},\ \text{tip\_hash})$ is irreflexive, total and transitive; the A>B>C>A cycle resolves to B>C>A
P2	The winner at the close is independent of the order in which candidates were submitted (verified over all permutations)
P3	When the message queue is non-empty: a provisional landing does not change what is canonical; a heavier candidate can be verified against the same frozen baseline and supersede it; the close commits the winner's outcome exactly once; the cursor is monotone with no double consumption
P5	A lazy close does not change the winner; after the close, a candidate can only open the next window
P6	Window state is a pure function of L1 history; after truncation by a shallow reorg, replay is consistent and reorged-out candidates disappear atomically
P7	Under the shared gas budget plus per-entry admission, a legal block exists for all 300 randomly generated reachable queue states (including non-genesis cursors); the maximum prefix under both constraints respects the caps
P7b	**The per-message gas cap is load-bearing** with no forced queue it is the only thing standing between the chain and "no legal block" — an over-cap entry would stall m_consumed permanently, so enqueue must refuse it
P9	The setter invariants are cross-checked between **independently declared deployment values** ($\Delta\_\text{lag,final} \geq \text{prov+W\_settle\_max}$, $\text{D\_anchor\_max} \geq$ the worst-case path, $\text{W\_settle} \geq$ proving + inclusion); causal ordering $\text{anchor.L1\_time} \leq \text{L2\_time(slot)}$ (an explicit time base, including the equality boundary)
P10	Slashing effectiveness is scoped to **settlement windows**: a slashing applies to a window iff it became effective strictly before that window opened, so the slash set freezes with the baseline and every candidate in a window is judged alike, whatever its landing time. A mid-window effectiveness applies from the next window, a delay of at most one W_settle. The superseded landing-time rule is pinned in the model against the same inputs
P11	Fallback eligibility is snapshotted the moment the window opens on $\text{lag\_final} > \Delta\_\text{lag,final}$ and is held until the window closes; a short provisional candidate that resets lag_prov to zero does not revoke eligibility
P12	Enqueueing in mid-window: the queue is append-only and referenced by sequence number, and the baseline freezes only the cursor and state — the outcome of earlier candidates is unchanged, a heavier candidate can consume the new entries deterministically, and the moment of enqueueing does not affect the verification result

Property	Content
P13	Fallback reimbursement: C_fixed is metered per $(count, tip\_slot)$ **level**, one per level, paid at close to the level's holder, plus φ_adv per marginal block and φ_data per published byte consumed. Every improver recovers its cost; a tip_hash grind collapses to one level and one C_fixed. Both superseded rules — the shared per-window cap and per-improvement metering — are pinned against the same inputs; verified over multiple submitters, over adversarial sequences alternating count-improving and tip-only candidates, and over a 40-candidate tip_hash grind

the total-order key, the window state machine, cursor arithmetic, gas shares and timing geometry)., anchor geometry and causal ordering, the moment a slashing takes effect, and the fallback accounting snapshot are in the model (P9–P11); the storage layout and gas cost of the Solidity-level $acceptCandidate$ entry point are not modelled here and remain an implementation concern (§11). The base-fee share (§6.5 φ_land) is an economic action of the accounting at the close that changes neither window state nor cursor semantics, and is not yet in the model. **final acceptance requires a human safety review** — the model and the review document are a gate, not a sign-off.